



ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

Μελέτη των Πρωτοκόλλων RPL και RPL-Lite για το Διαδίκτυο των Πραγμάτων

Σπουδαστής:
Χαράλαμπος ΔΗΜΑΡΙΔΗΣ

Υπεύθυνος καθηγητής:
Ελευθέριος ΜΑΜΑΤΑΣ

30 Σεπτεμβρίου 2020

ΕΛΛΗΝΙΚΗ
ΔΗΜΟΚΡΑΤΙΑ
Πανεπιστήμιο Μακεδονίας

ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

**Μελέτη των Πρωτοκόλλων RPL
και RPL-Lite για το Διαδίκτυο
των Πραγμάτων**

Μελέτη των Πρωτοκόλλων RPL και RPL-Lite για το Διαδίκτυο των Πραγμάτων

του Χαράλαμπου Δημαρίδη

Σε αυτό το σημείο θα ήθελα να ευχαριστήσω τον κ. Ελευθέριο Μαμάτα, επιβλέπων καθηγητή της πτυχιακής μου εργασίας, και τον κ. Γεώργιο Βιολέτα, καθηγητή του Κολεγίου Εφαρμοσμένης Μηχανικής του Ριάντ, για την υποστήριξη και την πολύτιμη βοήθεια που έλαβα κατά την διάρκεια της διατριβής μου. Τέλος, θα ήθελα να ευχαριστήσω το οικογενειακό μου περιβάλλον για την ψυχολογική υποστήριξη που μου πρόσφεραν καθ' όλη την διάρκεια του ταξιδιού αυτού.

Περίληψη

Με το RPL πρωτόκολλο έχουμε πλέον την δυνατότητα να δημιουργήσουμε δίκτυα έξυπνων συσκευών που απαιτούν ελάχιστη κατανάλωση ενέργειας για το Διαδίκτυο των Πραγμάτων. Εξίσου σημαντική όμως είναι και η αξιοπιστία του δικτύου εκτός από την κατανάλωση ενέργειας. Κάνοντας χρήση ενός νέου λειτουργικού συστήματος προσομοίωσης δικτύων χαμηλής ισχύος, ονόματι Contiki-NG, θα συγκρίνουμε το πρωτόκολλο RPL με μια μινιμαλιστική έκδοση του πρωτοκόλλου, το RPL-Lite. Θα ξεκινήσουμε εντρυφώντας στον τρόπο λειτουργίας του RPL πρωτοκόλλου και της ελαφριάς έκδοσης του, και θα αναδείξουμε τα καινούργια χαρακτηριστικά που φέρνει το Contiki-NG σε σχέση με την παλαιότερη έκδοση, το Contiki 3.0. Έπειτα θα περάσουμε στην ανάλυση παρόμοιων ερευνών με σχέση το RPL και τέλος με την βοήθεια δύο μετρικών θα περάσουμε στην ανάλυση και διεξαγωγή συμπερασμάτων σχετικά με την απόδοση των πρωτοκόλλων που εξετάζουμε.

Περιεχόμενα

Περιεχόμενα	I
Κατάλογος Σχημάτων	III
1 Εισαγωγή	1
2 Υπόβαθρο της μελέτης	4
2.1 Εισαγωγή	4
2.2 Internet of Things	4
2.2.1 RPL: το πρωτόκολλο για το Διαδίκτυο των Πραγ- μάτων	4
2.2.2 Δίκτυα ασυρμάτων αισθητήρων	7
2.2.3 Πρωτόκολλο των δικτύων χαμηλής ισχύος και κι- νητικότητα κόμβων	7
2.3 Τοπολογίες του RPL πρωτοκόλλου	8
2.3.1 DODAG	8
2.3.2 Επίπεδα (Rank)	9
2.4 Μηνύματα ελέγχου	10
2.5 Λειτουργίες του πρωτοκόλλου RPL	12
2.5.1 Υποστηριζόμενα μοτίβα κυκλοφορίας	12
2.5.2 Τρόποι λειτουργίας	12
2.5.3 Upward Routes	12
2.5.4 Downward Routes, RPL-traffic-delivery	13
2.5.5 Ανίχνευση και αποφυγή βρόχου	14
2.6 Χρονομετρητής Trickle	14
2.7 Ανακεφαλαίωση	15
3 Τα περιβάλλοντα Contiki και Contiki-NG	17
3.1 Εισαγωγή	17

3.2	Κύριες λειτουργίες του Contiki-NG	18
3.2.1	RPL-lite	21
3.2.2	6LoWPAN	21
3.2.3	6TiSCH	22
3.2.4	CoAP	22
3.2.5	MQTT	22
3.3	Σύγκριση των RPL εκδόσεων στο Contiki-ng	23
3.3.1	Πλεονεκτήματα του RPL-lite σε σχέση με το RPL- classic	23
3.3.2	Μειονεκτήματα του RPL-lite σε σχέση με το RPL- classic	23
3.4	Ανακεφαλαίωση	24
4	Άλλες μελέτες	25
5	Μεθοδολογία	28
6	Πειραματική ανάλυση	35
6.1	Πρώτο σενάριο: Συχνή επικοινωνία για διαφορετικές τοπολογίες	35
6.2	Δεύτερο σενάριο: Επικοινωνία ανά λεπτό για διαφορε- τικές τοπολογίες	38
6.3	Τρίτο σενάριο: Μέτρια επικοινωνία για διαφορετικές τοπολογίες	41
6.4	Τέταρτο σενάριο: Αραιή επικοινωνία για διαφορετικές τοπολογίες	44
6.5	Πέμπτο σενάριο: Πολύ αραιή επικοινωνία για διαφορε- τικές τοπολογίες	46
7	Συμπεράσματα	49
8	Προτάσεις για μελλοντικές μελέτες	51
	Βιβλιογραφία	53

Κατάλογος Σχημάτων

2.1	Βασική Αρχιτεκτονική των WSN	5
2.2	Παράδειγμα του Διαδικτύου των Πραγμάτων	6
2.3	Γράφος DODAG	9
2.4	Επίπεδα γράφου	10
2.5	Διάγραμμα Μηνυμάτων ελέγχου	11
2.6	Μοτίβα κυκλοφορίας	13
2.7	Αλγόριθμος Trickle για κάθε κόμβο	15
3.1	Γραφική διασύνδεση Cooja	20
5.1	10, 25 και 50 κόμβοι στους άξονες x και y	29
5.2	Βιβλιοθήκη ενεργοποίησης του uip statistics	30
5.3	Τροποποίηση του αρχείου Makefile	31
5.4	10, 15 και 20 κόμβοι στους άξονες x και y	33
6.1	Διαγράμματα Μέσου Όρου του Control Overhead πρώτου σεναρίου	36
6.2	Διαγράμματα Μέσου Όρου του Packet Delivery Ratio πρώτου σεναρίου	38
6.3	Διαγράμματα Μέσου Όρου του Control Overhead δευ- τέρου σεναρίου	39
6.4	Διαγράμματα Μέσου Όρου του Packet Delivery Ratio δευτέρου σεναρίου	41
6.5	Διαγράμματα Μέσου Όρου του Control Overhead τρίτου σεναρίου	42
6.6	Διαγράμματα Μέσου Όρου του Packet Delivery Ratio τρίτου σεναρίου	43
6.7	Διαγράμματα Μέσου Όρου του Control Overhead τε- τάρτου σεναρίου	45

6.8	Διαγράμματα Μέσου Όρου του Packet Delivery Ratio τετάρτου σεναρίου	46
6.9	Διαγράμματα Μέσου Όρου του Control Overhead πέμ- πτου σεναρίου	47
6.10	Διαγράμματα Μέσου Όρου του Packet Delivery Ratio πέμπτου σεναρίου	48

Κεφάλαιο 1

Εισαγωγή

Εν έτη 2020 παρατηρούμε μια ραγδαία εξέλιξη σε τεχνολογίες οι οποίες χρησιμοποιούνται σε δίκτυα, όπως ραγδαία εξέλιξη υπάρχει και στον τομέα του Διαδικτύου των Πραγμάτων (Internet of Things). Το Διαδίκτυο των Πραγμάτων είναι ένας τομέας της Πληροφορικής όπου τα τελευταία χρόνια έχει αρχίσει να ανθίζει, καθώς με την εξέλιξη των πρωτοκόλλων δικτύων που παρατηρούμε, μπορούν να επιτευχθούν οι στόχοι της συγκεκριμένης τεχνολογίας. Σκοπός αυτής της τεχνολογίας είναι όλες οι συσκευές μας να είναι «έξυπνες», συνδεδεμένες σε ένα δίκτυο, και να συλλέγουν πληροφορίες όπου θα τις στέλνουν μέσω αυτού του δικτύου σε έναν κεντρικό διακομιστή που θα επεξεργάζεται αυτά τα δεδομένα.

Τέτοιου είδους δίκτυα αποτελούνται κυρίως από διάφορους αισθητήρες που τοποθετούμε με σκοπό την συλλογή των δεδομένων, οι οποίοι απαιτούν χαμηλή κατανάλωση ενέργειας, έτσι ώστε να παραμένουν ενεργά τα δίκτυα αυτά για μεγάλα χρονικά διαστήματα. Συνεπώς, για την επικοινωνία των αισθητήρων, που αποτελούν κόμβους (nodes) ενός τέτοιου δικτύου, θα πρέπει να χρησιμοποιήσουμε και κατάλληλα πρωτόκολλα επικοινωνίας, έτσι ώστε να ικανοποιήσουμε τις προϋποθέσεις λειτουργίας των κόμβων. Καταλυκτικό παράγοντα αποτελεί το πρωτόκολλο δρομολόγησης για δίκτυα χαμηλής ισχύος (RPL - Routing Protocol for Low-Power and Lossy Networks) το οποίο αναπτύχθηκε για αυτό τον σκοπό.

Στόχος της διατριβής είναι η σύγκριση διαφορετικών εκδόσεων και υλοποιήσεων του πρωτοκόλλου RPL. Για την επίτευξη της έρευνας μας, αξιοποιήσαμε ένα νέο λειτουργικό σύστημα προσομοίωσης δικτύων,

το περιβάλλον Contiki-NG, το οποίο αποτελεί μια νέα πρόταση του λειτουργικού συστήματος προσομοίωσης δικτύων Contiki, προσφέρει δυνατότητα χρήσης της επίσημης εκδοχής του πρωτοκόλλου RPL, ενώ παράλληλα προτείνει μια νέα έκδοση του πρωτοκόλλου RPL, την έκδοση RPL-Lite, η οποία είναι μια ελαφρύτερη υλοποίηση του πρωτοκόλλου. Πάνω σε αυτές τις δύο εκδόσεις λοιπόν, αποφασίσαμε να δουλέψουμε και να διαπιστώσουμε εάν αυτή η νέα πρόταση πρωτοκόλλου δρομολόγησης πακέτων για δίκτυα χαμηλής ισχύος έχει να προσφέρει κάτι παραπάνω από τη επίσημη εκδοχή του.

Επιπλέον, αποφασίσαμε να ερευνήσουμε τα παραπάνω καθώς, πρώτον με τα χρόνια οι τεχνολογίες του διαδικτύου των πραγμάτων έχουν αρχίσει να ωριμάζουν και να χρησιμοποιούνται όλο ένα και περισσότερο σε σενάρια καθημερινής χρήσης. Δεύτερον το RPL πρωτόκολλο είναι υπεύθυνο για την δρομολόγηση των πακέτων σε δίκτυα του Διαδικτύου των Πραγμάτων, οπότε είναι βασικός κρίκος στην λειτουργία τους. Τρίτων δεν υπάρχει ακόμη έρευνα σύγκρισης της νέας πρότασης του πρωτοκόλλου που φέρνει το λειτουργικό σύστημα Contiki-NG με την επίσημη έκδοση, όπως επίσης υπάρχουν ελάχιστες έρευνες με την χρήση του ίδιου του Contiki-NG.

Η μεθοδολογία που ακολουθήθηκε για να επιτευχθεί ο στόχος της διατριβής μας είναι η εξής. Αρχικά μελετήσαμε τη σχετική βιβλιογραφία, ώστε να κατανοήσουμε σε βάθος το RPL, όπως και τις διαφορετικές εκδοχές λειτουργίας του με αποθήκευση πληροφοριών τοπολογίας (storing) και χωρίς (non-storing). Η σχετική ανάλυση επικεντρώνεται στις θεωρητικές πλευρές του πρωτοκόλλου και βρίσκεται στο κεφάλαιο 2 (Υπόβαθρο της Μελέτης). Το κεφάλαιο καταλήγει σε παρατηρήσεις για τα πλεονεκτήματα και μειονεκτήματα των βασικών εκδόσεων του RPL. Στην συνέχεια ελέγξαμε διαφορετικές υλοποιήσεις του RPL στα περιβάλλοντα Contiki και Contiki-NG, αλλά και τα τελευταία. Σημειώνουμε ότι το Contiki-NG αποτελεί καινούρια πρόταση όπως αναφέρθηκε και παραπάνω, μετεξέλιξη του Contiki, και υπάρχουν ελάχιστες μελέτες που το αξιοποιούν. Το βήμα αυτό σχετίζεται με την πρακτική διάσταση των εκδόσεων του RPL και περιλαμβάνει το RPL-Lite, μια ελαφριά μινιμαλιστική υλοποίηση του RPL, ενώ η αντίστοιχη ανάλυση περιλαμβάνεται στο κεφάλαιο 3 (Περιβάλλοντα Contiki και Contiki-NG). Στόχος του αντίστοιχου κεφαλαίου είναι η αναζήτηση πλεονεκτημάτων και μειονεκτημάτων διαφορετικών υλοποιήσεων του RPL, μέσα από μια παρουσίαση των

Contiki και Contiki-NG.

Έπειτα περάσαμε στην μελέτη συναδέλφων οι οποίοι χρησιμοποίησαν είτε την απλή έκδοση του Contiki είτε την έκδοση του Contiki-NG χρησιμοποιώντας παρόμοιες μετρικές έτσι ώστε να διαπιστώσουν πλεονεκτήματα και μειονεκτήματα διαφορετικών εκδόσεων του πρωτοκόλλου RPL ή τροποποιημένων αλγορίθμων που κάνει χρήση το ίδιο το πρωτόκολλο. Στόχος αυτού του κεφαλαίου είναι η ύπαρξη ενός μέτρου σύγκρισης της διατριβής με μελέτες άλλων συναδέλφων. Η σχετική ανάλυση περιλαμβάνεται στο κεφάλαιο 4 (Άλλες μελέτες).

Από τη στιγμή που καλύψαμε τη θεωρητική πλευρά της διατριβής πραγματοποιήσαμε πειραματική ανάλυση που συγκρίνει τις υλοποιήσεις RPL-Classic και RPL-Lite, μέσω κατάλληλης μεθοδολογίας, η οποία παρουσιάζεται αναλυτικά στο κεφάλαιο 5 (Μεθοδολογία). Στόχος αυτού του κεφαλαίου είναι η ανάδειξη του τρόπου με τον οποίο διεξήγαμε την έρευνα, όπως και τα προβλήματα τα οποία αντιμετωπίσαμε έτσι ώστε ερευνητές που θα συνεχίσουν την έρευνα πάνω σε αυτή την νέα πρόταση του πρωτοκόλλου, αλλά και του λειτουργικού συστήματος Contiki-NG, να είναι σε θέση να αντιμετωπίσουν άμεσα παρόμοια προβλήματα με την βοήθεια του παρόντος.

Μεταβαίνοντας στο κεφάλαιο 6 (Πειραματική Ανάλυση) σχολιάζονται αναλυτικά τα εμπειρικά αποτελέσματα που εξάγαμε έχοντας στόχο να αναδείξουμε και επαληθεύσουμε βασικά πλεονεκτήματα και μειονεκτήματα των διαφορετικών εκδόσεων και υλοποιήσεων του RPL. Τέλος, στο κεφάλαιο 7 (Συμπεράσματα) κάνουμε μια ανακεφαλαίωση των ευρημάτων αυτής της έρευνας και στο κεφάλαιο 8 (Ιδέες για μελλοντικές μελέτες) προτείνουμε σενάρια τα οποία δεν είχαμε τον χρόνο να εξετάσουμε. Κυριότερος στόχος, όμως, αυτού του κεφαλαίου είναι η πρόταση ιδεών για τη λύση κάποιων προβλημάτων που αντιμετωπίσαμε κατά την διάρκεια της πειραματικής ανάλυσης, τα οποία αναφέρονται στο κεφάλαιο 5 της μεθοδολογίας, καθώς δεν καταφέραμε να τα λύσουμε μέσω της παρούσης διατριβής λόγω χρονικών περιορισμών.

Κεφάλαιο 2

Υπόβαθρο της μελέτης

2.1 Εισαγωγή

Στο κεφάλαιο που ακολουθεί θα μελετήσουμε το πρωτόκολλο RPL έτσι ώστε να κατανοήσουμε σε βάθος τις λειτουργίες του για την ανάγκη της έρευνας. Το πρωτόκολλο RPL μπορεί να το συναντήσει κάποιος στην ανάπτυξη εφαρμογών του Διαδικτύου των Πραγμάτων (Internet of Things), καθώς είναι απαραίτητο για την δρομολόγηση δικτυακών πακέτων μεταξύ των κόμβων δικτύων χαμηλής ισχύος. Η δεύτερη ενότητα του κεφαλαίου, λοιπόν, παρουσιάζει τα ασύρματα δίκτυα αισθητήρων καθώς και τις ιδιαίτερες απαιτήσεις τους, τα οποία χαρακτηρίζονται κυρίως από περιορισμένους υπολογιστικούς πόρους και αυστηρές απαιτήσεις σε κατανάλωση ενέργειας. Στις επόμενες ενότητες του κεφαλαίου, παρουσιάζουμε και αναλύουμε βασικές λειτουργίες του πρωτοκόλλου RPL, το οποίο στοχεύει να καλύψει τις παραπάνω απαιτήσεις.

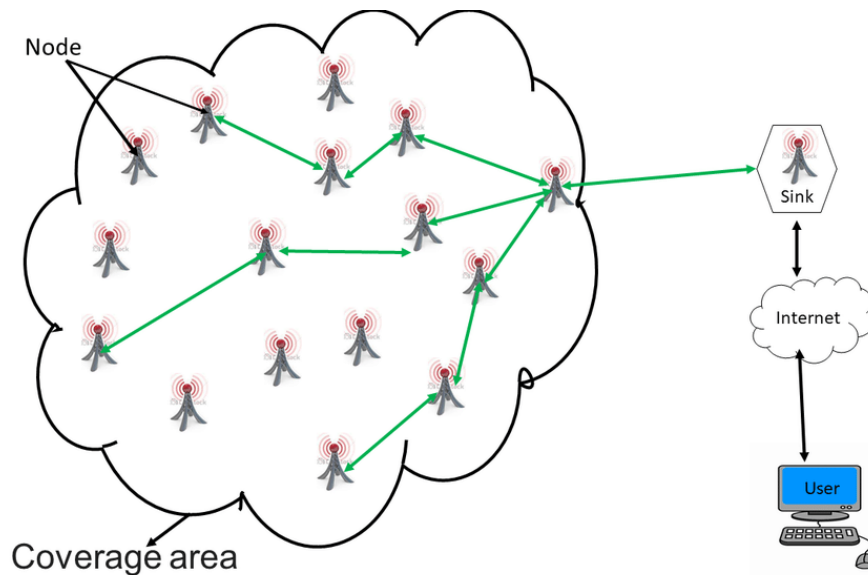
2.2 Internet of Things

2.2.1 RPL: το πρωτόκολλο για το Διαδίκτυο των Πραγμάτων

Για να μπορέσουμε να κατανοήσουμε τις βασικές λειτουργίες του πρωτοκόλλου RPL και για πιο λόγο είναι τόσο σημαντικό, θα πρέπει να γίνει μια εξήγηση σε ποιο τομέα της Επιστήμης της Πληροφορικής είναι χρήσιμο. Σίγουρα, ένα μεγάλο ποσοστό ανθρώπων στον πλανήτη Γη έχει ακούσει τον όρο Διαδίκτυο των Πραγμάτων ή και I-

Internet of Things - IoT. Τι είναι, όμως το Διαδίκτυο των Πραγμάτων; Στο Διαδίκτυο των Πραγμάτων όλες οι συσκευές επικοινωνούν μεταξύ τους μέσω δικτύου, και αποτελούνται από αισθητήρες ελάχιστων απαιτήσεων και περιορισμένων δυνατοτήτων, έτσι ώστε να είναι οικονομικές και να έχουν περιορισμένη κατανάλωση ενέργειας.

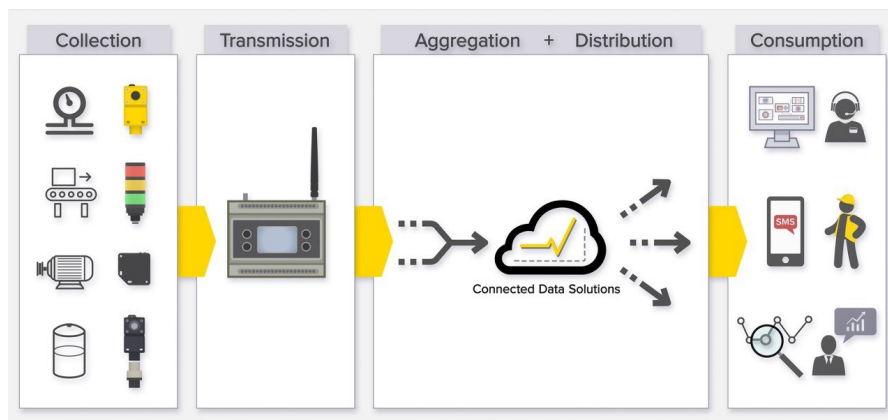
Συνήθως, οι αισθητήρες επικοινωνούν μέσω ενός τοπικού ασύρματου δικτύου, το οποίο ονομάζεται δίκτυο ασυρμάτων αισθητήρων (WSN - Wireless Sensor Network) (βλ. σχήμα 2.1), με τα δεδομένα που συλλέγουν να τα στέλνουν σε έναν κεντρικό διακομιστή, γνωστό και ως root ή sink. Στη συνέχεια, ένας root μπορεί να συνδεθεί με τον έξω κόσμο, δηλαδή το διαδίκτυο και να σχηματιστούν είτε ένα, είτε πολλά ίδιου τύπου δίκτυα, τα οποία θα αποστέλλουν τις πληροφορίες που συλλέγουν σε μια βάση δεδομένων. Η βάση δεδομένων με την σειρά της μπορεί να αξιοποιήσει αλγορίθμους τεχνητής νοημοσύνης, που απαιτούν αρκετά δεδομένα, έτσι ώστε ο τελικός χρήστης να λαμβάνει χρήσιμες πληροφορίες. Τα WSN μπορούν να συνοψιστούν και ως Δίκτυα Χαμηλής Ισχύος (LLNs - Low-Power and Lossy Networks)



Σχήμα 2.1: Βασική Αρχιτεκτονική των WSN

Αναλυτικότερα, οι ασύρματοι αισθητήρες, χρησιμοποιούν συχνά το πρωτόκολλο δρομολόγησης πακέτων RPL - Routing Protocol for Low-Power and Lossy Networks για να στείλουν τα δεδομένα που συλλέγουν στον root, με τον τρόπο που εξηγήσαμε παραπάνω. Στην συνέχεια ο root, στην περίπτωση που θέλουμε αυτά τα δεδομένα να προωθηθούν σε κάποιο διακομιστή, αναλαμβάνει να τα προωθήσει σε έναν δρομολογητή συνόρου (LBR - LLN border router) [8], και αυτός με τη σειρά του, με την χρήση του Διαδικτύου να τα προωθήσει στον server. Έπειτα ο server θα αναλάβει την επεξεργασία των δεδομένων και την αποστολή των αποτελεσμάτων στον τελικό χρήστη (βλ. σχήμα 2.2).

Η έρευνα μας επικεντρώνεται στις λειτουργίες δρομολόγησης των Δικτύων Ασυρμάτων Αισθητήρων, οπότε θα περιοριστούμε στα όρια των αντίστοιχων δικτύων (LLNs) και όχι στο υπόλοιπο κομμάτι του Διαδικτύου, όπως π.χ. η έρευνα [7]. Συνεπώς, θα εξετάσουμε την απόδοση αποστολής πακέτων από τους κόμβους (nodes) προς τον κεντρικό κόμβο συλλογής δεδομένων (sink node ή root) και αντιστρόφως με την χρήση του πρωτοκόλλου RPL [6].



Σχήμα 2.2: Παράδειγμα του Διαδικτύου των Πραγμάτων

2.2.2 Δίκτυα ασυρμάτων αισθητήρων

Για να γίνει αποδοτικά σε μεγάλη κλίμακα, η συλλογή δεδομένων, με τη βοήθεια αισθητήρων, είτε από έμβια όντα, είτε από το περιβάλλον στο οποίο ζούμε σε πραγματικό χρόνο, χρειάζεται ένα πρωτόκολλο το οποίο θα είναι σχεδιασμένο κατάλληλα, έτσι ώστε να μην υπάρχει απώλεια δεδομένων. Επιπλέον, το δίκτυο αυτό θα πρέπει καταναλώνει όσο το δυνατόν λιγότερη ενέργεια είναι εφικτό, έτσι ώστε στην περίπτωση που χρησιμοποιούμε μπαταρίες ενέργειας να επεκτείνουμε όσο περισσότερο μπορούμε τη διάρκεια ζωής ενός αισθητήρα. Εκτός από τα παραπάνω, το πρωτόκολλο που θα χρησιμοποιηθεί, θα πρέπει να είναι εύκολα προγραμματιζόμενο, επεκτάσιμο, έτσι ώστε να είναι εύκολη η χρήση πολλών αισθητήρων ή η προσθήκη επιπλέον αισθητήρων, να παρέχει ασφάλεια στη μετάδοση των δεδομένων και φυσικά οι αισθητήρες να έχουν χαμηλό κόστος. Συνεπώς, για να επιτευχθούν τα παραπάνω χαρακτηριστικά θα πρέπει να χρησιμοποιηθεί υλικό το οποίο θα έχει περιορισμένες δυνατότητες επεξεργαστικής ισχύος, μνήμης, αποθηκευτικού χώρου και κατανάλωσης ενέργειας [1, 2, 10]. Και σε αυτό το κομμάτι γίνεται η προσθήκη του RPL πρωτοκόλλου, το οποίο παρέχει μηχανισμούς δρομολόγησης, από και προς τους αισθητήρες, σε ασύρματα δίκτυα αισθητήρων [3, 4].

2.2.3 Πρωτόκολλο των δικτύων χαμηλής ισχύος και κινητικότητα κόμβων

Όπως αναφέρθηκε και παραπάνω, οι αισθητήρες με τους οποίους θα πραγματοποιηθεί η συλλογή τέτοιου είδους δεδομένων ανήκουν σε δίκτυα χαμηλής ισχύος, για να μπορέσουν να είναι αποδοτικοί στην δουλειά που κάνουν. Για να επιτευχθεί λοιπόν αυτό, δημιουργήθηκε το RPL - IPv6 routing protocol for low-power and lossy networks έτσι ώστε να εφαρμοστεί σε LLNs - low-power and lossy networks. Παρακάτω, θα γίνει μια εισαγωγή του πρωτοκόλλου, έτσι ώστε ο αναγνώστης να μπορεί να κατανοήσει το βασικό υπόβαθρο και την ορολογία της διατριβής.

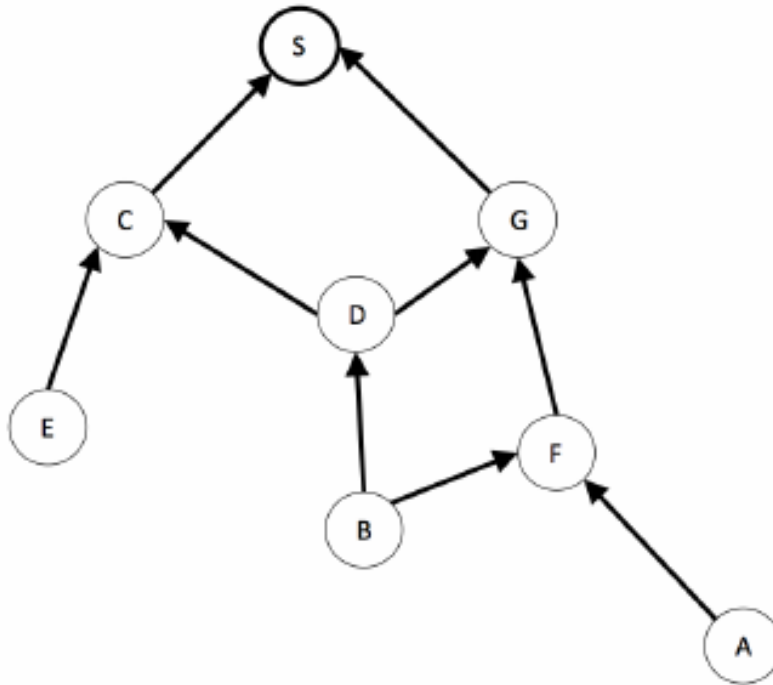
Εκτός από τους σταθερούς κόμβους, όπως για παράδειγμα έναν αισθητήρα καπνού, σε ένα WSN μπορούν να υπάρχουν και κινούμενοι κόμβοι, π.χ. αισθητήρες που συλλέγουν πληροφορίες από ένα έμβιο οργανισμό, οι οποίοι στέλνουν δεδομένα μέσω ασυρμάτου δικτύου σε ένα root. Η κινητικότητα δημιουργεί την ανάγκη πρωτοκόλλων

που μπορούν να διαχειριστούν δυναμικές τοπολογίες που χαρακτηρίζονται από πολλές επαναμεταδόσεις πακέτων και διακοπτόμενη επικοινωνία. Για αυτό τον λόγο στο πρωτόκολλο RPL προτείνονται τρόποι βελτιστοποίησης αποστολής των πακέτων έτσι ώστε να μην υπάρχει απώλεια πακέτων κατά την αποστολή των δεδομένων [11, 18]. Επιπλέον έρευνα περί των κινητών κόμβων γίνεται στο [9], όπου αναλύονται προβλήματα και λύσεις που σχετίζονται με την κινητικότητα των κόμβων. Τα ζητήματα κινητικότητας δεν περιλαμβάνονται στην διατριβή, δηλαδή χρησιμοποιούμε στην ανάλυση μας μόνο σταθερούς κόμβους.

2.3 Τοπολογίες του RPL πρωτοκόλλου

2.3.1 DODAG

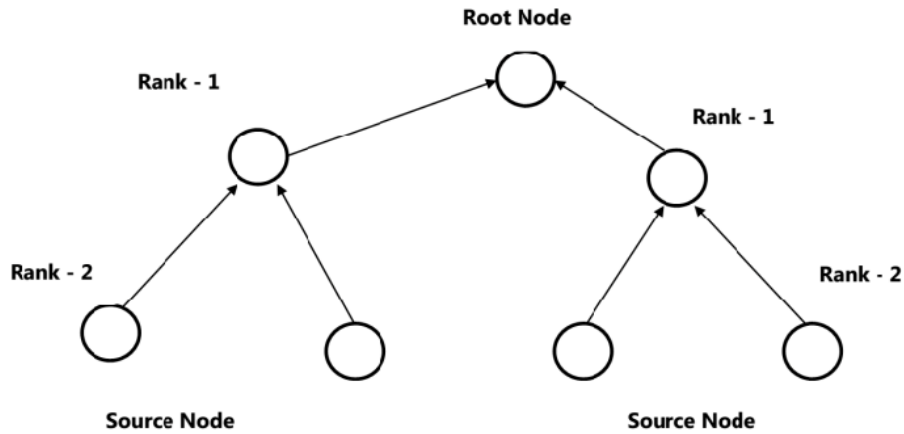
Το RPL είναι βασισμένο σε στην τοπολογία των DAG - Directed Acyclic Graph, δηλαδή στο να φτιαχτεί ένα δένδρο από γράφους, το οποίο είναι κλασικό και ένας κόμβος δεν συσχετίζεται με πολλούς parents αλλά μόνο με έναν. Το πρωτόκολλο RPL οργανώνει τους κόμβους του δικτύου σαν ένα DODAG - Destination-Oriented DAG δρομολογώντας τους κόμβους σε ένα κεντρικό sink ή root, το οποίο είναι το μοναδικό που μπορεί να αρχικοποιήσει το DODAG. Το DODAG είναι βασισμένο σε ανταλλαγή μηνυμάτων ελέγχου (Control messages) μέσω του root. Μια μικρή ανάλυση των μηνυμάτων ελέγχου θα γίνει παρακάτω (βλ. Ενότητα 2.5) [12, 15, 20, 5, 14]. Περισσότερα για τη διαδικασία κατασκευής ενός DODAG στο [17].



Σχήμα 2.3: Γράφος DODAG

2.3.2 Επίπεδα (Rank)

Με το επίπεδο rank (βλ. σχήμα 2.4) μπορούμε να προσδιορίσουμε που βρίσκεται ένας κόμβος μέσα στο γράφο, σε σχέση με τους άλλους κόμβους, και τον κόμβο ρίζα (root node), από πάνω προς τα κάτω. Ο τρόπος υπολογισμού του επιπέδου εξαρτάται από το DAGs Objective Function (OF) [20].



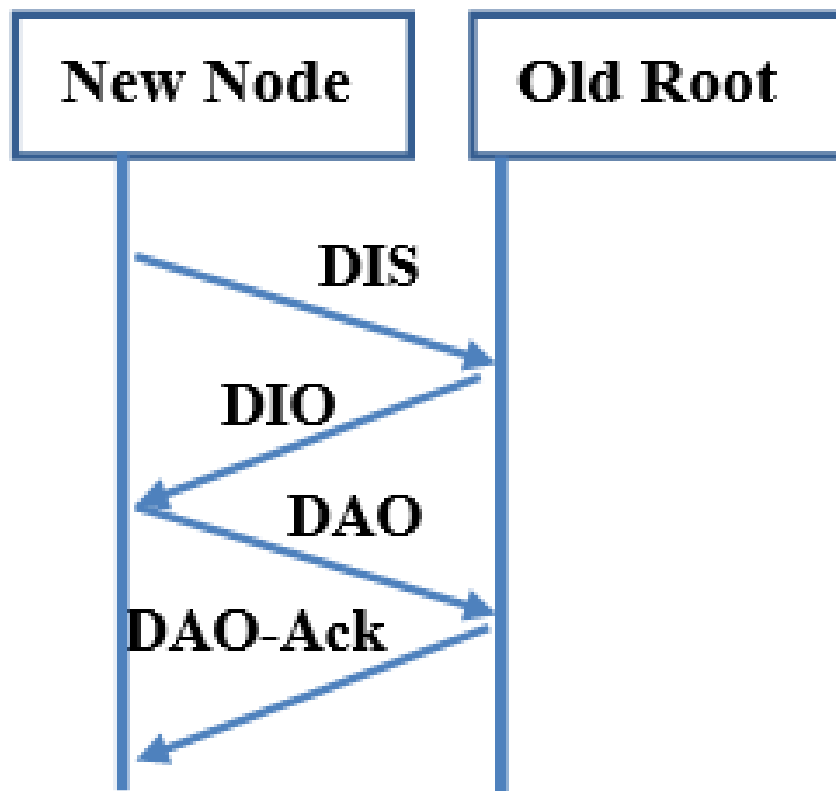
Σχήμα 2.4: Επίπεδα γράφου

2.4 Μηνύματα ελέγχου

Όπως αναφέρθηκε παραπάνω υπάρχουν τα μηνύματα ελέγχου (Control messages) (βλ. σχήμα 2.5) που αναλαμβάνουν την ενημέρωση του γράφου κατά την αποστολή πακέτων μεταξύ των κόμβων. Υπάρχουν 5 κύρια είδη μηνυμάτων ελέγχου:

- **DIO: DODAG Information Object:** Το μήνυμα DIO μεταφέρει πληροφορίες και επιτρέπει έναν κόμβο να ανακαλύψει μια RPL οντότητα, να μάθει τις παραμέτρους αυτής της οντότητας, να επιλέξει parent σε ένα DODAG και ο κόμβος αυτός να παραμείνει στο DODAG.
- **DIS: DODAG Information Solicitation:** Το μήνυμα DIS χρησιμοποιείται από τους κόμβους για να ζητήσουν από ένα γείτονα κόμβο μήνυμα DIO.
- **DAO: Destination Advertisement Object:** Το μήνυμα DAO χρησιμοποιείται στη διάδοση πληροφοριών προορισμού προς τον root (βλ. ενότητα 2.5.4) κατά μήκος του DODAG. Όταν είναι ενεργοποιημένη η λειτουργία storing (βλ. ενότητα 2.5.2), το DAO αποστέλλει πακέτα δεδομένων σε συγκεκριμένους parents. Όταν είναι ενεργοποιημένη η λειτουργία non-storing (βλ. ενότητα 2.5.2), το DAO αποστέλλει πακέτα δεδομένων στο DODAG root.

- DAO-ACK: Destination Advertisement Object Acknowledgement: Το μήνυμα DAO-ACK στέλνεται ως ένα πακέτο δεδομένων από έναν παραλήπτη DAO ως απάντηση.
- CC: Consistency Check: Το μήνυμα CC μήνυμα χρησιμοποιεί στον έλεγχο του μετρητή μηνυμάτων ασφαλείας και σε απαντήσεις ζητημάτων προκλήσεων. Σημείωση: ένα CC μήνυμα αποστέλλεται ως ασφαλές μήνυμα RPL.



Σχήμα 2.5: Διάγραμμα Μηνυμάτων ελέγχου

2.5 Λειτουργίες του πρωτοκόλλου RPL

2.5.1 Υποστηριζόμενα μοτίβα κυκλοφορίας

Τα κύρια υποστηριζόμενα μοτίβα κυκλοφορίας πακέτων μέσα σε έναν γράφο DODAG είναι τα point-to-point (P2P), point-to-multipoint (P2MP) και multipoint-to-point (MP2P). Με τον όρο P2P εννοούμε την αποστολή μηνυμάτων από έναν κόμβο σε έναν άλλον είτε είναι ο κόμβος είναι ρίζα αυτός, είτε όχι. Από την άλλη το P2MP είναι όταν ένας κόμβος στέλνει ένα πακέτο σε πολλούς κόμβους. Πιο συνηθισμένο φαινόμενο είναι το MP2P στο οποίο πολλοί κόμβοι στέλνουν σε ένα κόμβο και αυτό συμβαίνει στη περίπτωση που όλα τα "παιδιά" του γράφου στέλνουν στη ρίζα (root) του γράφου. Τέλος, το RPL σχεδιάστηκε για να λειτουργεί όσο πιο σωστά γίνεται με το MP2P. Για να λειτουργήσει το MP2P κάνει χρήση των μηνυμάτων ελέγχου DIO και DIS, ενώ για την λειτουργία των μοτίβων P2P και P2MP πρέπει να χρησιμοποιηθούν τα μηνύματα DAO και DAO-ACK [15, 20].

2.5.2 Τρόποι λειτουργίας

Υπάρχουν δύο τρόποι λειτουργίας στο πρωτόκολλο RPL στην περίπτωση της προς-τα-κάτω downward δρομολόγησης, όπου απαρτίζεται από τα P2P και P2MP μοτίβα λειτουργίας. Αυτοί οι δύο τρόποι είναι με αποθήκευση storing και χωρίς αποθήκευση non-storing. Αναλυτικότερα:

- τρόπος με αποθήκευση (storing mode): στην προκειμένη περίπτωση όταν ένα παιδί στέλνει ένα μήνυμα ελέγχου προς τον πατέρα, ο πατέρας αποθηκεύει αυτό το μήνυμα πριν στείλει το δικό του
- τρόπος χωρίς αποθήκευση (non-storing mode): σε αυτή την περίπτωση, όταν ένα παιδί στέλνει ένα μήνυμα ελέγχου μέσω unicast έχοντας προορισμό την ρίζα του DODAG, οι κόμβοι από τους οποίους θα περάσει για να αποσταλλεί το μήνυμα δεν θα το αποθηκεύουν [15, 14].

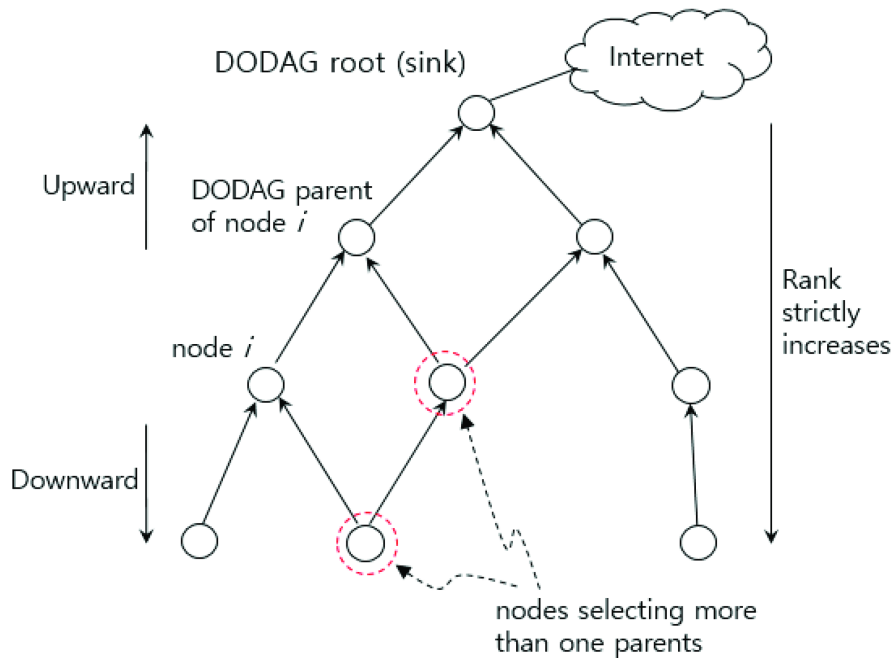
2.5.3 Upward Routes

Με τον όρο upward routing εννοούμε τον τρόπο δρομολόγησης πακέτων από ένα φύλο του δένδρου προς την ρίζα δηλαδή «προς τα

επάνω» (βλ. σχήμα 2.6). Κύριος μηχανισμός που έχει σχεδιαστεί για να κάνει όσο καλύτερα γίνεται αυτή την λειτουργία είναι το MP2P communication, δηλαδή πολλοί κόμβοι να στέλνουν σε έναν κεντρικό κόμβο, τη ρίζα του δένδρου. Αυτό, μπορεί κάποιος να το συναντήσει σε περιπτώσεις όπου κόμβοι μαζεύουν διάφορες πληροφορίες από το περιβάλλον και στέλνουν αυτές τις πληροφορίες κάπου κεντρικά[20, 19].

2.5.4 Downward Routes, RPL-traffic-delivery

Από την άλλη πλευρά σε αυτή την περίπτωση, έχουμε τις λειτουργίες P2P και P2MP, καθώς ένας κόμβος δεν αναλαμβάνει να στείλει το πακέτο προς την ρίζα, αλλά προς κάποιον άλλον κόμβο, και σε αυτή την περίπτωση μπορεί να γίνει συνδυασμός με το storing mode. Είναι δηλαδή ο τρόπος να στέλνουν οι κόμβοι δεδομένα «προς τα κάτω» (βλ. σχήμα 2.6). Φυσικά και οι δύο τρόποι routing μπορούν να συνδυαστούν χάρη στη δυνατότητα ενσωμάτωσης πολλαπλών τοπολογιών μέσα στο RPL [14].



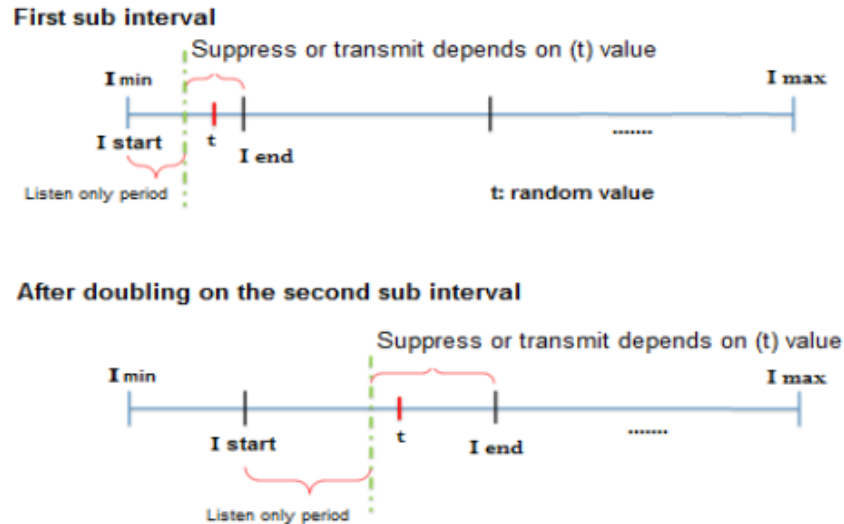
Σχήμα 2.6: Μοτίβα κυκλοφορίας

2.5.5 Ανίχνευση και αποφυγή βρόχου

Σε αντίθεση με τα κανονικά δίκτυα υπολογιστών, όπου πολλές φορές μπορεί να δημιουργηθεί κάποιος βρόχος περιστασιακά στην περίοδο αλλαγής κάποιας τοπολογίας, στα LLNs πρέπει να αποφευχθεί ο βρόχος το γρηγορότερο δυνατό, καθώς μπορεί να καταλήξει στην απώλεια ενός πακέτου. Για αυτό τον λόγο έχουν γίνει αρκετές προτάσεις τέτοιου είδους μηχανισμών για ενσωμάτωση στο πρωτόκολλο RPL, π.χ. ο κανόνας μέγιστου βάθους (max_depth rule) [15].

2.6 Χρονομετρητής Trickle

Ένας πολύ βασικός αλγόριθμος του RPL είναι ο χρονομετρητής trickle. Βασίζεται στη λογική βαθμιαίας εξασθένησης της ανανέωσης της τοπολογίας, σε περίπτωση που εντοπιστεί στατικός γράφος, προκειμένου να σωθεί ενέργεια. Κύριος σκοπός του είναι η διαχείριση της μετάδοσης των πακέτων έτσι ώστε να μην υπάρχουν συγκρούσεις πακέτων μέσα σε ένα WSN. Το παραπάνω το πετυχαίνει με δύο λειτουργίες: 1. όταν εντοπίζεται ασυνέπεια μέσα στο δίκτυο, αυξάνεται ο ρυθμός ελέγχου επιτρέποντας το δίκτυο να επανέλθει σε φυσιολογικό επίπεδο, 2. Στην περίπτωση υπάρχουν πακέτα που κυκλοφορούν στο δίκτυο χωρίς κάποιο λόγο, επαναλαμβάνει το ίδιο μήνυμα μέχρι όλοι οι κόμβοι να μην το χρειάζονται. Ο αλγόριθμος, αρχικοποιεί το κύριο διάστημα μεταξύ των κόμβων ξεκινώντας από το I_{min} και τελειώνοντας στο I_{max} , όπου I_{min} και I_{max} είναι μεταβλητές. Στη συνέχεια, δημιουργεί στο κύριο διάστημα υποδιαστήματα, και κάθε υποδιάστημα ξεκινάει από το I_{start} και τελειώνει στο I_{end} , όπου I_{start} και I_{end} είναι μεταβλητές. Η μεταβλητή I_{start} I_{min} και η μεταβλητή I_{end} $I_{start} * 2$ [16, 20].



Σχήμα 2.7: Αλγόριθμος Trickle για κάθε κόμβο

Ο αλγόριθμος έχει 3 βασικές παραμέτρους:

1. I_{max} : μέγιστο μέγεθος διαστήματος
2. I_{min} : ελάχιστο μέγεθος διαστήματος
3. K : σταθερά πλεονασμού

Τρεις μεταβλητές που διατηρούνται από τον αλγόριθμο είναι:

1. I : μέγεθος του διαστήματος
2. C : μετρητής
3. T : συγκεκριμένο χρονικό διάστημα μέσα σε ένα διάστημα

2.7 Ανακεφαλαίωση

Στο δεύτερο κεφάλαιο, λοιπόν, είδαμε με ποιο κομμάτι του IoT θα ασχοληθούμε, ποια είναι η χρήση πρωτοκόλλου σε δίκτυα χαμηλής ισχύος, όπως και τι δυνατότητες προσφέρει σε αυτά τα δίκτυα. Αναλύσαμε τις βασικές λειτουργίες του πρωτοκόλλου RPL για να κατανοήσουμε τον τρόπο λειτουργίας του και έτσι να προχωρήσουμε

παρακάτω στην έρευνα μας κατανοώντας τις παραμέτρους που θα τροποποιήσουμε για τις ανάγκες της ανάλυσης μας. Στο επόμενο κεφάλαιο θα εξετάσουμε το πειραματικό περιβάλλον που θα χρησιμοποιήσουμε στη διατριβή, το οποίο επιπρόσθετα υποστηρίζει μια μινιμαλιστική, πιο ελαφριά έκδοση του πρωτοκόλλου RPL.

Κεφάλαιο 3

Τα περιβάλλοντα Contiki και Contiki-NG

3.1 Εισαγωγή

Στο τρίτο κεφάλαιο θα ασχοληθούμε με την ανάλυση των λειτουργιών του εργαλείου που θα χρησιμοποιήσουμε στη διατριβή. Πρόκειται για μια νέα έκδοση του υπάρχοντος λογισμικού προσομοίωσης δικτύων Contiki, το Contiki-NG, το οποίο εκτός από αρκετές καινούργιες λειτουργίες που φέρνει (π.χ. ενσωματώνει νέα πρωτόκολλα όπως το 6TiSCH), προτείνει μια ελαφρύτερη έκδοση του RPL πρωτοκόλλου, το RPL Lite. Παρακάτω, στην ενότητα που ακολουθεί, θα αναλύσουμε τις κύριες λειτουργίες του πειραματικού περιβάλλοντος Contiki-NG, το οποίο αποτελεί και ολοκληρωμένο λειτουργικό σύστημα για συσκευές του Διαδικτύου των Πραγμάτων.

Περνώντας στη συνέχεια της τρίτης ενότητας, θα διευκρινίσουμε ποιες είναι οι κύριες διαφορές μεταξύ των Contiki και Contiki-NG, το τελευταίο είναι στην ουσία μια νέα βελτιωμένη έκδοση του πρώτου. Αρκετή υλοποίηση του Contiki-NG ξεκίνησε από την αρχή, με τις επιμέρους υποενότητες να αναλύουν και να παρουσιάζουν κάποιες καινούργιες λειτουργίες, καθώς και υποστήριξη των νέων πρωτοκόλλων και λειτουργιών που φέρνει το Contiki-NG. Τέλος, στην τέταρτη ενότητα, θα γίνει μια σύγκριση μεταξύ των δύο εκδόσεων του RPL πρωτοκόλλου που υποστηρίζει το λειτουργικό Contiki-NG, δηλαδή της κλασικής έκδοσης, η οποία έχει αναλυθεί στο κεφάλαιο 2, με την ελαφριά έκδοση, δηλαδή το RPL Lite.

Ο σκοπός αυτού του κεφαλαίου είναι να αναδείξουμε ένα καινούργιο, ανανεωμένο λειτουργικό σύστημα σε σχέση με τον προκάτοχο του. Θέλουμε να δούμε τις καινούργιες λειτουργίες που φέρνει, όπου είναι και ο λόγος που επιλέξαμε να το χρησιμοποιήσουμε. Επιπλέον, μας ενδιαφέρει να εξετάσουμε τις καίριες διαφορές των δυο εκδόσεων του πρωτοκόλλου δρομολόγησης πακέτων RPL, πριν περάσουμε στην συγκριτική πειραματική ανάλυση της διατηρηβής ανάμεσα σε διαφορετικές εκδόσεις του RPL, με σκοπό να διαπιστώσουμε εάν αυτές οι διαφορές θα παρατηρηθούν και εκεί.

3.2 Κύριες λειτουργίες του Contiki-NG

Το ανοιχτό λογισμικό Contiki είναι ένα εργαλείο που βοηθάει στην υλοποίηση ανάπτυξης εφαρμογών για το Διαδίκτυο των Πραγμάτων. Κάνοντας χρήση της γλώσσας προγραμματισμού C και έχοντας την δυνατότητα ο χρήστης να προσομοιώσει ένα δίκτυο, χρησιμοποιώντας την εφαρμογή που ανέπτυξε, μπορεί να βοηθήσει σε πολλούς τομείς πάνω στην ανάπτυξη και εφαρμογή του IoT. Το λειτουργικό είναι ανοιχτού κώδικα και μπορεί το βρει κάποιος στο GitHub Repository. Κύριος σκοπός του λειτουργικού είναι η μετάφραση προγραμμάτων από γλώσσα C σε εκτελέσιμο κώδικα, και στην συνέχεια η εκτέλεση του προγράμματος με την χρήση κάποιου mote και κατάλληλου περιβάλλοντος εξομοίωσης. Ας ξεκινήσουμε όμως με το τι είναι ένα mote. Με τον όρο mote εννοούμε το είδος του υλικού που θα χρησιμοποιήσει κάποιος για να τρέξει ένα πρόγραμμα το οποίο προορίζεται για το Διαδίκτυο των πραγμάτων και συγκεκριμένα εφαρμογών WSN. Με αυτό τον τρόπο, αντί κάποιος να αγοράσει το υλικό και να εκτελέσει τον κώδικα που έχει γράψει πάνω σε αυτό, με την βοήθεια του Contiki-NG μπορεί να εξομοιώσει αυτό το υλικό και να δει την συμπεριφορά του προγράμματος που δημιουργήσε. Αυτό βοηθάει στην περίπτωση που ένα είδος υλικού δεν είναι κατάλληλο για ένα έργο, να μπορεί να αλλάξει χωρίς να υπάρχει οικονομικό αντίκτυπο. Στην έκδοση που υπάρχει ανεβασμένη στο GitHub υπάρχουν κάποια motes, αλλά φυσικά μπορεί κάποιος να προσθέσει κάποιο δικό του τύπο, αρκεί να είναι συμβατό.

Εκτός από τα παραπάνω, ο χρήστης μπορεί με την αλλαγή παραμέτρων, μέσω των βιβλιοθηκών που έχουν σχεδιαστεί για το συγκεκριμένο λογισμικό, να παραμετροποιήσει τον τρόπο λειτουργίας των

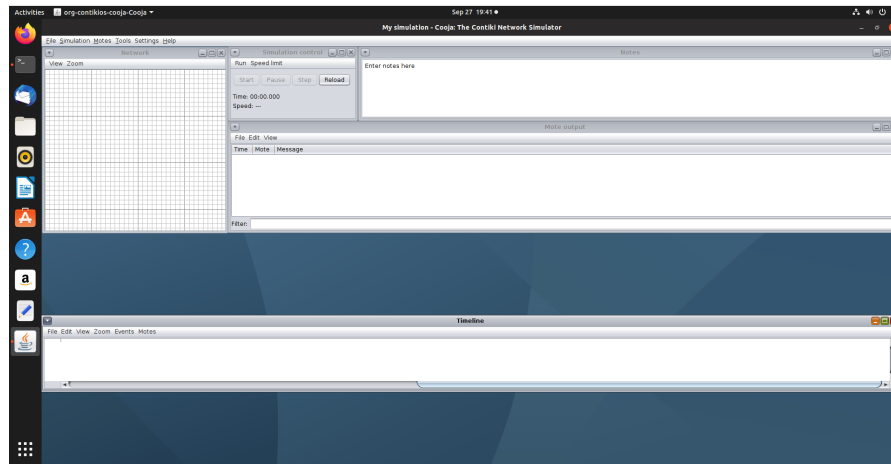
προγραμμάτων του, έτσι ώστε να δουλεύει έτσι όπως ακριβώς θέλει ο δημιουργός του. Για παράδειγμα, μπορεί να παραμετροποιήσει τον αριθμό των upward μηνυμάτων μέσω μεταβλητής σε μια από τις βιβλιοθήκες, καθώς εξ ορισμού μπορεί να στέλνει upward μηνύματα μέχρι σε δεκαέξι κόμβους. Επιπλέον λειτουργίες που έχει το Contiki-NG είναι εργαλεία καταγραφής πακέτων, μηνυμάτων, ενέργειας, καθώς και το νέο σύστημα καταγραφής log file που έχει προστεθεί, με σκοπό ο χρήστης να μπορεί να παρακολουθεί σε όποιο επίπεδο επιθυμεί την κατάσταση του υλικού, του δικτύου κτλ. Εδώ πρέπει να επισημανθεί ότι παρόμοια εργαλεία χρησιμοποιεί και το Contiki, με τα δυο λειτουργικά να έχουν κάποια διαφορετικά πακέτα παρακολούθησης και διαφορετικό τρόπο σύνταξης στις βιβλιοθήκες.

Για την εκτέλεση των πειραμάτων κάνουμε χρήση του Cooja Simulator, ο οποίος έχει δημιουργηθεί σε γλώσσα Java. Μέσω αυτού μπορεί ένας προγραμματιστής να εκτελέσει τον κώδικα που έχει γράψει, να επιλέξει τι είδος mote θα χρησιμοποιήσει, πόσους κόμβους θα έχει το δίκτυο του, ποια θέση στον χώρο θα έχουν οι κόμβοι, πόσα μέτρα θα εκπέμπουν ασύρματο σήμα και σε γενικές γραμμές να παραμετροποιήσει ένα δίκτυο ασύρματων αισθητήρων όπως ακριβώς επιθυμεί. Ο Cooja Simulator μπορεί να τρέξει είτε με γραφική διασύνδεση (βλ. σχήμα 3.1), είτε μέσω command line, χωρίς γραφική διασύνδεση, έχοντας την δυνατότητα ο χρήστης να συντάξει κατάλληλα script για να μπορεί να εκτελέσει παραδείγματα το ένα μετά το άλλο, για συγκεκριμένο χρονικό διάστημα, έτσι ώστε να μπορέσει να αυτοματοποιήσει όσο το δυνατόν περισσότερο την εκτέλεση πειραμάτων.

Για το λειτουργικό Contiki υπάρχουν δύο βασικές υλοποιήσεις την δεδομένη περίοδο. Και οι δύο υλοποιήσεις είναι ανοιχτού κώδικα και οι εφαρμογές που μπορούν να αναπτυχθούν σε αυτές είναι μέσω της γλώσσας προγραμματισμού C. Το Contiki OS ξεκίνησε ως πρόγραμμα ανοιχτού κώδικα τον Μάρτιο του 2003 και αυτή την στιγμή βρίσκεται στην 3.0 έκδοση. Από την άλλη πλευρά, το Contiki-NG είναι και αυτό λογισμικό ανοιχτού κώδικα, και αυτήν την στιγμή βρίσκεται στην έκδοση 4.5, η οποία δημοσιοποιήθηκε 14/05/2020. Και τα δύο λογισμικά μπορεί να τα βρει κάποιος στο GitHub Repository.

Το Contiki-NG, σε σχέση με τον προκάτοχο του Contiki, από την αρχή του σχεδιασμού και της ανάπτυξης του, έδωσε μεγάλη έμφαση στην ενσωμάτωση αξιόπιστων πρωτοκόλλων για δίκτυα χαμηλής κατανάλωσης ενέργειας όπως είναι το IPv6/6LoWPAN - IPv6 over Low-

Power Wireless Personal Area Network , το 6TiSCH - IPv6 over the Timeslotted Channel Hopping, το RPL - Routing Protocol for Low-Power and Lossy Networks, το CoAP - Constraint Application Protocol και το MQTT - Message Queue Telemetry Transport. Επιπλέον, το Contiki-NG διαθέτει αναλυτικό οδηγό εγκατάστασης, εκτελέσιμα παραδείγματα και σελίδες τεκμηρίωσης των περισσότερων λειτουργιών των πρωτοκόλλων που υποστηρίζει μέσα στο GitHub Repository του λογισμικού από τους δημιουργούς, εκτός από την υποστήριξη τρίτων χρηστών [32, 37, 38, 25].



Σχήμα 3.1: Γραφική διασύνδεση Cooja

3.2.1 RPL-lite

Η υλοποίηση του RPL-Lite ξεκίνησε στοχεύοντας, πρώτον στην αποφυγή της πολυπλοκότητας του απλού RPL και δεύτερον στη μείωση της κατανάλωσης ενέργειας. Υπάρχουν αρκετές υλοποιήσεις του RPL-Lite, όμως σε αυτή την μελέτη θα εξετάσουμε την υλοποίηση την οποία χρησιμοποιεί το Contiki-NG και είναι ενσωματωμένη μέσα στο λειτουργικό. Για να λειτουργήσει το RPL-Lite πρωτόκολλο, το οποίο είναι βασισμένο στην οργάνωση μιας τοπολογίας DAG, και με την μετάδοση μηνυμάτων σε ένα δυναμικό δίκτυο μέσω του Trickle Timer, οι κόμβοι μπορούν να εργάζονται αυτόνομα καθώς με τον παραπάνω τρόπο ενεργοποιείται η ελάχιστη διαμόρφωση τους. Εδώ πρέπει να σημειωθεί ακόμη ότι, το πρωτόκολλο δεν έχει σχεδιαστεί για να δουλεύει σε μια μιρφή αρχιτεκτονικής, όπως για παράδειγμα το IP, αλλά μπορεί να δουλέψει σε όποια μορφή είναι απαραίτητο.

Σε αντίθεση με το κλασικό RPL, στο RPL-Lite το μόνο που χρειάζεται να γνωρίζει ένας κόμβος, είναι ο γείτονας που μπορεί να στείλει πακέτα και για αυτόν το λόγο δεν υπάρχουν DAGIDs - Directed Acyclic Graph IDs στο πρωτόκολλο. Επιπροσθέτως, όπως και στο κλασικό πρωτόκολλο RPL, έτσι και στο RPL-Lite υποστηρίζεται η προς-τα-επάνω (upwards) αποστολή δεδομένων στην ρίζα, καθώς και η προς-τα-κάτω (downwards), με χρήση μηνυμάτων INDA - IPv6 Inverse Neighbor Discovery Advertisement, μόνο που ως επιλογή του πρωτοκόλλου είναι η λειτουργία non-storing, με τη λειτουργία storing να μην υποστηρίζεται. Τέλος, το RPL-Lite κάνει χρήση μόνο των DAG Information Advertisement Messages for Upward Routing μηνυμάτων ελέγχου, τα οποία στην ουσία είναι μια ελαφρύτερη έκδοση των DIO μηνυμάτων ελέγχου, με τους υπόλοιπους τύπους μηνυμάτων ελέγχου να μην υπάρχουν για χρήση. [20].

3.2.2 6LoWPAN

Το πρωτόκολλο 6LoWPAN δημιουργήθηκε για μεταφορά πακέτων μικρού μεγέθους και δίκτυα χαμηλής ισχύος μέσω του πρωτοκόλλου IPv6. Η υλοποίηση του ξεκίνησε από την αρχή με το σκεπτικό ότι θα χρησιμοποιηθεί σε πολύ μικρά δίκτυα με περιορισμένες δυνατότητες. Στην μελέτη του [21] μπορεί να βρει κάποιος αναλυτική εξήγηση της αρχιτεκτονικής του πρωτοκόλλου.

3.2.3 6TiSCH

Με το μεγάλο κόστος που διακατέχει η καλωδίωση εγκαταστάσεων έτσι ώστε συσκευές να μπορούν να επικοινωνούν μεταξύ τους, ξεκίνησε η προσπάθεια η ασύρματη σύνδεση συσκευών μέσω του IPv6 protocol να γίνεται το νέο πρότυπο. Έτσι, το 2013 ξεκίνησε μέσω της IETF μια ομάδα ανάπτυξης του IPv6 over the Timeslotted Channel Hopping (6TiSCH). Το 6TiSCH είναι ένα πρωτόκολλο το οποίο χρησιμοποιείται κυρίως σε χαμηλής ισχύος ασύρματα δίκτυα καταγραφής δεδομένων (monitoring) εφαρμογές. Καθώς σκοπός της παρούσης εργασίας είναι διαφορετικός, οι [22] εξηγούν αναλυτικά στην μελέτη τους την αρχιτεκτονική του πρωτοκόλλου αυτού.

3.2.4 CoAP

Το μοντέλο Constraint Application Protocol (CoAP) λαμβάνει την λειτουργία πελάτη/διακομιστή όπως και στο HTTP πρωτόκολλο, αλλά σε αντίθεση με το HTTP ανταλλάσει μηνύματα ασύγχρονα μέσω ενός datagram-oriented transport όπως είναι το UDP protocol και βρίσκεται ανάμεσα στο application layer και στο UDP. Τέλος, το CoAP ανταλλάσει τέσσερις τύπους μηνυμάτων: Confirmable, Non-confirmable, Acknowledgement, Reset. Περαιτέρω για το CoAP πρωτόκολλο μπορεί να βρει κάποιος στην μελέτη των [23] και των [24].

3.2.5 MQTT

Μια μικρή αναφορά θα γίνει και στο MQTT πρωτόκολλο, το οποίο λειτουργεί στο application layer και “συνομιλεί” με το Transmission Control Protocol (TCP) και το IP protocol. Χρησιμοποιεί μια αρχιτεκτονική Topic-Based Publish-Subscribe, που σημαίνει ότι, όταν ένας client κάνει publish ένα μήνυμα M σε ένα θέμα T, τότε όλοι οι client που είναι subscribed στο θέμα T, θα λάβουν το μήνυμα M [8]. Περαιτέρω για την το MQTT αναφέρονται στην μελέτη των [24].

3.3 Σύγκριση των RPL εκδόσεων στο Contiki-ng

3.3.1 Πλεονεκτήματα του RPL-lite σε σχέση με το RPL-classic

Ένα από τα κύρια πλεονεκτήματα του RPL-Lite πρωτοκόλλου είναι ότι σχεδιάστηκε έτσι ώστε να είναι ενεργειακά πιο αποδοτικό από την κλασική έκδοση του πρωτοκόλλου, έτσι ώστε δίκτυα χαμηλής ισχύος να έχουν περισσότερη αντοχή στον χρόνο. Δεύτερο πλεονέκτημα, το οποίο ταυτόχρονα είναι και μειονέκτημα, και θα εξηγηθεί στην επόμενη υποενότητα αυτής της ενότητας, είναι ότι ο κώδικας του πρωτοκόλλου καταναλώνει λιγότερο αποθηκευτικό χώρο, καθώς είναι πιο συρρικνωμένος, σε σχέση και πάλι με την κλασική έκδοση. Αυτό σημαίνει θεωρητικά, από την στιγμή που καταναλώνει λιγότερο αποθηκευτικό χώρο, μπορούν να χρησιμοποιηθούν συσκευές οι οποίες έχουν μικρότερο αποθηκευτικό χώρο, και στην ουσία κάθε συσκευή να κοστίζει λιγότερο σε χρήματα. Τρίτο πλεονέκτημα, και σύμφωνα με όσα ειπώθηκαν στην ενότητα 3.2.2., κάθε κόμβος μπορεί να λειτουργήσει αυτόνομα.

3.3.2 Μειονεκτήματα του RPL-lite σε σχέση με το RPL-classic

Βέβαια, εκτός από τα πλεονεκτήματα που έχει η χρήση του RPL-Lite πρωτοκόλλου, υπάρχουν και μειονεκτήματα, και αυτά είναι κυρίως οι μειωμένες λειτουργίες του πρωτοκόλλου, λόγω της συρρίκνωσης που υποβλήθηκε ο κώδικας για να καταλαμβάνει λιγότερο αποθηκευτικό χώρο. Πρώτο μειονέκτημα είναι η στέρηση της λειτουργίας Global Repair, η οποία κάνει επανεκκίνηση όλο τον γράφο. Σε αυτή την περίπτωση, δίνεται η δυνατότητα να πραγματοποιηθεί η επανεκκίνηση μέσω λίστας από trigger events, αλλά αποτελεί δυνατότητα υλοποίησης και δεν έχει ακόμα ενσωματωθεί στο πρωτόκολλο. Δεύτερο μειονέκτημα είναι ότι δεν υπάρχει ένας τοπικός DAG. Αντίθετα, κάθε φορά που ένας κόμβος δεν υπάρχει μέσα στο γράφο, απλά προστίθεται μέσω των γειτόνων. Τρίτο μειονέκτημα είναι η μη-ύπαρξη κάποιου μηχανισμού ασφαλείας, βασίζεται σε εξωτερικούς μηχανισμούς. Τέλος, ένα ακόμη μειονέκτημα είναι οι λιγότεροι τύποι μηνυμάτων ελέγχου, σε σχέση με την κλασική έκδοση του πρωτοκόλλου, γεγονός που συνεπάγεται μειωμένη λειτουργικότητα.

3.4 Ανακεφαλαίωση

Όπως παρατηρούμε, το λειτουργικό Contiki-NG είναι μια καινούργια πρόταση πάνω στον τομέα ανάπτυξης εφαρμογών για το Διαδίκτυο των Πραγμάτων με πολλές νέες λειτουργίες και ενσωματωμένα πρωτόκολλα στοχεύοντας σε χαμηλή κατανάλωση ενέργειας. Επιπλέον, η νέα ελαφρύτερη έκδοση του RPL πρωτοκόλλου φαίνεται αρκετά υποσχόμενη για δίκτυα χαμηλής ισχύος. Αυτό που μένει να διαπιστώσουμε εάν όντως αποδίδει καλύτερα από την κλασσική έκδοση.

Κεφάλαιο 4

Άλλες μελέτες

Ολοκληρώνοντας τη θεωρητική ανασκόπηση, στο κεφάλαιο που ακολουθεί θα κάνουμε μια βιβλιογραφική επισκόπηση σε έρευνες συναδέλφων πάνω στο πρωτόκολλο RPL και τις παραλλαγές του. Στόχος μας είναι να έχουμε ένα φάσμα πληροφοριών πριν ξεκινήσουμε τη δική μας έρευνα, έτσι ώστε να ακολουθήσουμε μια σωστή μεθοδολογία κατά την διάρκεια της διεξαγωγής των πειραμάτων μας με σκοπό να παράξουμε αξιόπιστα αποτελέσματα. Επιπλέον, θέλουμε να δείξουμε ότι μέχρι και σήμερα χρησιμοποιούνται οι παλαιότερες εκδόσεις του Contiki σε έρευνες σχετικές με τη δική μας, και όχι το Contiki-NG, κατά ένα μεγάλο ποσοστό.

Ξεκινώντας, η έρευνα [26] έχει ως στόχο την εξέταση μιας νέας υλοποίησης του αλγορίθμου trickle timer, με όνομα “Dynamic Double Trickle Timer Algorithm”, σε σχέση με τη βασική έκδοση του αλγορίθμου. Ως πειραματικό εργαλείο χρησιμοποιήθηκε ο Cooja με την χρήση του λογισμικού Contiki 2.7. Χρησιμοποιήθηκαν σενάρια των 20 και 40 κόμβων, με κάθε σενάριο να τρέχει με 3 διαφορετικές τιμές RX - Reception Ratio των 20, 60 και 100 και μετρήθηκαν ο χρόνος σύγκλισης, η κατανάλωση ενέργειας και η αναλογία παράδοσης πακέτων (PDR - Packet Delivery Ratio). Από άποψη PDR, ο δυναμικός αλγόριθμος φαίνεται ότι έχει καλύτερα αποτελέσματα σε σχέση με τη βασική έκδοση και στους 20 και στους 40 κόμβους, με εξαίρεση τους 20 κόμβους με τιμή $RX = 20$. Ακριβώς το ίδιο συμπέρασμα φαίνεται να υπάρχει και για την κατανάλωση ενέργειας. Τέλος, ο δυναμικός αλγόριθμος φαίνεται να είναι σε όλες τις περιπτώσεις καλύτερος, σε σχέση με τον βασικό, όσον αφορά την μετρική του χρόνου σύγκλισης.

Η έρευνα [27] στοχεύει στην αξιολόγηση της απόδοσης των αντικειμενικών λειτουργιών (Objective Functions) του RPL. Σε αυτή την εργασία μπορούμε να δούμε την εξέταση ανάμεσα σε δυο OF - Objective Functions, της OF0 και της MRHOF. Για να εξεταστούν χρησιμοποιήσαν τη μετρική του Packet Delivery Ratio καθώς και την κατανάλωση ενέργειας. Για την τοποθέτηση των κόμβων, οι ερευνητές χρησιμοποίησαν τις τοπολογίες random και grid που προσφέρει το λειτουργικό σύστημα Contiki μέσω του Cooja. Στην αρχή έγινε εξέταση του PDR με διάφορες τιμές RX και η έρευνα έδειξε όσο μεγαλύτερο είναι, τόσο μεγαλύτερο το PDR. Το ίδιο ισχύει και για την κατανάλωση ενέργειας, δηλαδή όσο ανέβαινε η τιμή του RX τόσο κατέβαινε η κατανάλωση ενέργειας. Τέλος, όσο αφορά τα OFs, το OF0 είναι πιο αποτελεσματικό από το MRHOF και στην παράδοση των πακέτων και στην κατανάλωση ενέργειας.

Συνεχίζοντας, περνάμε στην εργασία [28] με στόχο την ανάδειξη μιας δυναμικής έκδοσης του RPL, συγκρίνοντας την με μια φορητή mobile έκδοση της, καθώς και με την κλασική. Για την πραγματοποίηση των πειραμάτων, οι ερευνητές πραγματοποίησαν την έρευνα τους με την χρήση και πάλι του Cooja simulator στο Contiki 3.0. Από την διεξαγωγή αυτής της έρευνας, προκύπτει ότι το δυναμικό RPL είχε καλύτερη αναλογία παράδοσης πακέτων και κατανάλωση ενέργειας, με την φορητή έκδοση να έρχεται δεύτερη και την κλασική έκδοση να έχει την χειρότερη απόδοση.

Επόμενη έρευνα, η οποία μας ενδιαφέρει κατά ένα μεγάλο ποσοστό, καθώς δεν προλάβουμε να ασχοληθούμε στην παρούσα διατριβή, είναι η αξιολόγηση του RPL κάτω από συνθήκες κινητικότητας (mobility) η οποία δημοσιεύτηκε το 2012 [29]. Για την διεξαγωγή της μελέτης εξομοιώθηκαν 10 αυτοκίνητα ως κόμβοι τροποποιώντας την παράμετρο EXT για κάθε κόμβο. Οι αρθρογράφοι χρησιμοποίησαν, για να αξιολογήσουν τα παραπάνω, τις μετρικές του Control Overhead και του Packet Delivery Ratio. Σε γενικές γραμμές, όσο μικρότερη ήταν η περίοδος πρόβλεψης του EXT, τόσο και μεγαλύτερα ποσοστά υπήρχαν στον έλεγχο και την αναλογία παράδοσης πακέτων.

Μια ακόμη έρευνα που μας κέντρισε το ενδιαφέρον, και τυχαίνει να έχει παρόμοιους στόχους με τη δική μας έρευνα, είναι η [30] με σκοπό τη σύγκριση της βασικής έκδοσης του RPL με μια έκδοση του RPL πρωτοκόλλου η οποία ονομάζεται M-RPL - Multiple Path RPL. Σύμφωνα με τις μετρήσεις των ερευνητών, όπου χρησιμοποίησαν τον

Cooja Simulator με την χρήση του λειτουργικού συστήματος Contiki, από θέμα ροής δεδομένων, το M-RPL είναι ικανό για μεγαλύτερες ροές δεδομένων από την κανονική έκδοση, ενώ και στην κατανάλωση ενέργειας φαίνεται ότι έχει καλύτερη απόδοση.

Όπως και στην [29], έτσι και στην [31], εξετάζονται δύο πρωτόκολλα RPL με θέμα ένα δίκτυο οχημάτων (VANET - Vehicular Ad hoc Network) έτσι ώστε να διαπιστώσουν πιο πακέτο αποδίδει καλύτερα κάτω από συνθήκες κινητικότητας. Με τη βοήθεια των μετρικών PDR – Packet Delivery Ratio και Control Overhead καθώς και με την μέτρηση της καθυστέρησης μπόρεσαν να αξιολογήσουν το EXT-RPL - Expected Transmission Count in original RPL σε σχέση με το GI-RPL – Geographical Information for RPL. Από τις μετρήσεις μπορεί κάποιος να διαπιστώσει ότι σε θέμα παράδοσης πακέτων και καθυστέρησης το GI-RPL είναι καλύτερο σε σχέση με την άλλη έκδοση. Επιπλέον, από άποψη του ελέγχου, ναι μεν το GI-RPL έχει μικρότερο ποσοστό, αλλά στέλνει λιγότερα μηνύματα ελέγχου σε σχέση με την άλλη έκδοση και αυτά που στέλνει τα στέλνει με κάποια στρατηγική για να σώσει μηνύματα.

Επιπλέον παρατηρήθηκαν δημοσιεύσεις, σε επιστημονικά περιοδικά και επιστημονικά συνέδρια, έχοντας ως αντικείμενο να παρουσιάσουν τον τρόπο με τον οποίο λειτουργεί το Contiki. Δύο από αυτά ήταν τα [33] και [34], ενώ για το Contiki-NG βρήκαμε το βιβλίο [35] έχοντας το ίδιο αντικείμενο, και φυσικά σε μεγαλύτερη έκταση καθώς αναφερόμαστε σε βιβλίο. Τέλος, από το έτος 2018 παρατηρείται μια αύξηση των δημοσιεύσεων πάνω στον τομέα του IoT που κάνουν χρήση το Contiki-NG, όπως η έρευνα [36].

Ανακεφαλαιώνοντας, μπορούμε να δούμε ότι υπάρχουν πολλές μελέτες πάνω στο θέμα του Internet of Things με θέμα σύγκρισης το κλασσικό πρωτόκολλο του RPL και άλλες βελτιωμένες ή νέες ιδέες με σκοπό την αρίστευση της συμπεριφοράς ενός δικτύου, έτσι ώστε να υπάρχει αξιοπιστία. Πάντα μας ενδιαφέρει να έχουμε ένα αξιόπιστο δίκτυο με το χαμηλότερο κόστος, καθώς σε ορισμένες περιπτώσεις μπορεί αυτή η αξιοπιστία να σώσει ζωές, όπως για παράδειγμα ένα δίκτυο αισθητήρων καπνού σε ένα δάσος, ή σε ένα ορυχείο, ή σε μεγάλα κτήρια όπου στεγάζεται μεγάλος πληθυσμός ανθρώπων. Συνεπώς, το να μην έχουμε απώλεια πακέτων, καθώς και η ύπαρξη σωστής και γρήγορης δρομολόγησης είναι τα σημαντικά ζητήματα σε τέτοιες περιπτώσεις.

Κεφάλαιο 5

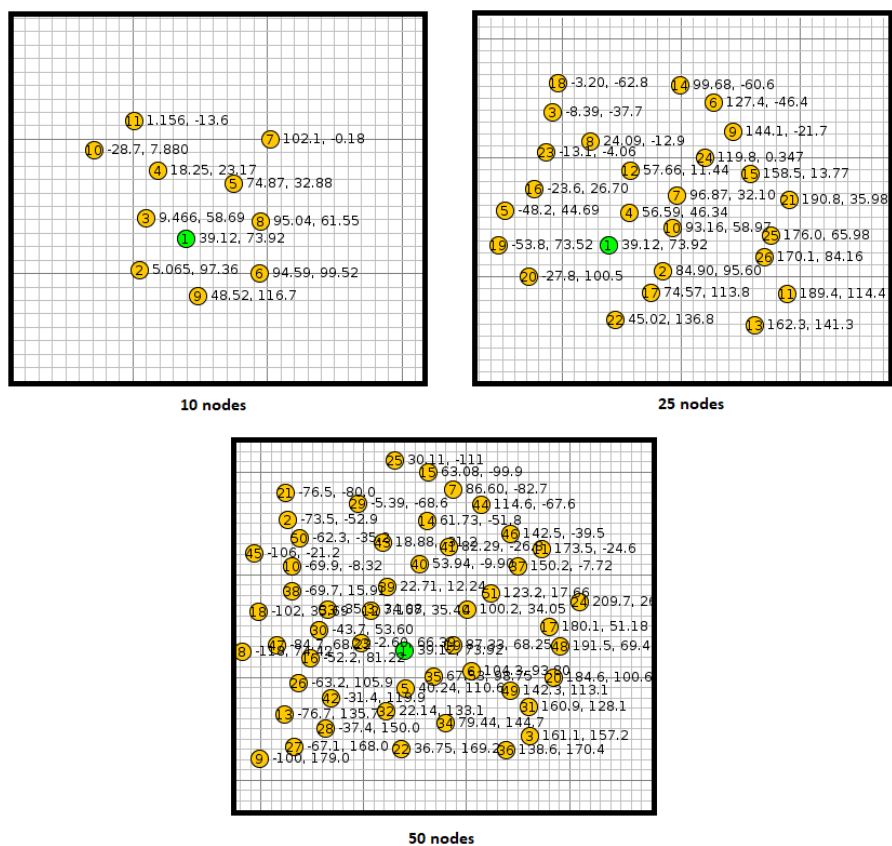
Μεθοδολογία

Στα κεφάλαια 2 και 3 αναλύσαμε το πρωτόκολλο που εξετάζουμε, το λογισμικό που θα χρησιμοποιήσουμε, μια πιο ελαφριά έκδοση που θα κάνουμε σύγκριση με την αρχική, καθώς και που μπορούμε να χρησιμοποιήσουμε τα παραπάνω εργαλεία και πρωτόκολλα σε πραγματικές συνθήκες. Στο κεφάλαιο που ακολουθεί θα αναλύσουμε για ποιον λόγο ξεκίνησε η διεξαγωγή της διατριβής μας, τον τρόπο με τον οποίο διεξήγαμε την έρευνα και ποια προβλήματα αντιμετωπίσαμε, έτσι ώστε έπειτα να είμαστε σε θέση να μεταβούμε στο πέμπτο κεφάλαιο στο οποίο γίνεται αναλυτική παρουσίαση των πειραματικών αποτελεσμάτων της έρευνας.

Ξεκινήσαμε έχοντας ως στόχο την εξέταση της απόδοσης δύο υλοποιήσεων του πρωτοκόλλου RPL με την χρήση του λειτουργικού συστήματος Contiki-NG. Η πρώτη είναι η κλασική υλοποίηση του πρωτοκόλλου, που αναλύσαμε στο δεύτερο κεφάλαιο, και θα εξετάσουμε την εκδοχή με storing λειτουργία καθώς και την εκδοχή με non-storing λειτουργία. Η δεύτερη έκδοση που θα εξετάσουμε είναι η ελαφριά (Lite) έκδοση του πρωτοκόλλου που εξηγήθηκε περεταίρω στην ενότητα 3.3.2. του τρίτου κεφαλαίου.

Πρωταρχικός στόχος της διατριβής είναι η θεωρητική και πειραματική σύγκριση εκδοχών του πρωτοκόλλου RPL, συμπεριλαμβάνοντας το RPL Lite, μέσω πέντε πειραματικών σεναρίων, θεωρώντας διαφορετικούς ρυθμούς αποστολής δεδομένων. Σε κάθε σενάριο επιθυμούμε να αξιολογήσουμε τρεις εκδοχές του πρωτοκόλλου, δηλαδή το RPL Classic με storing λειτουργία, το RPL Classic με non-storing λειτουργία και το RPL Lite ανάμεσα σε μικρό, μεσαίο και μεγάλο πλήθος

κόμβων. Έτσι λοιπόν, καταλήξαμε ότι το κάθε σενάριο θα αποτελε-
 ίται από τρία παραδείγματα των δέκα κόμβων, τρία παραδείγματα
 των εικοσιπέντε κόμβων και τρία παραδείγματα των πενήντα κόμ-
 βων, έχοντας κατά νου να καλύψουμε τις παραπάνω προϋποθέσεις
 (βλ σχήμα 5.1).



Σχήμα 5.1: 10, 25 και 50 κόμβοι στους άξονες x και y

Για τον σκοπό αυτό, έγινε χρήση ενός έτοιμου παραδείγματος που υπάρχει μέσα στο πακέτο του Contiki-NG, του rpl-udp, που στην ουσία είναι ένα πολύ απλό παράδειγμα echo server, με τους κόμβους να στέλνουν ένα upward message προς τον root, και αυτός με την σειρά του, να στέλνει ένα echo downward message προς τον κόμβο από όπου έλαβε το μήνυμα. Στο παράδειγμα έγινε προσθήκη κατάλληλων βιβλιοθηκών και κώδικα για την εκτύπωση μέσα σε log file πόσα udp πακέτα και μηνύματα ελέγχου στέλνει και λαμβάνει κάθε κόμβος, συμπεριλαμβανομένου του root, έτσι ώστε στη συνέχεια της έρευνας, τα συγκεκριμένα log files να επεξεργαστούν με την βοήθεια κάποιου λογισμικού.

Η τροποποίηση του κώδικα έγινε μονάχα για τον client, καθώς εάν χαθεί κάποιο πακέτο στη διαδρομή από τον client προς τον root, ο μόνος που θα μπορέσει να το αντιληφθεί είναι ο client. Πρώτα έγινε η δημιουργία μας βιβλιοθήκης με το όνομα “project-conf.h” και έγινε προσθήκη μέσω του “Makefile”, έτσι ώστε να μπορέσουμε να ενεργοποιήσουμε το εργαλείο uip statistics, το οποίο είναι ένα εργαλείο του Contiki-NG, και βοηθάει στην μέτρηση των control μηνυμάτων και των udp πακέτων που λαμβάνουν και αποστέλλουν οι κόμβοι. Στην εικόνα 5.2 μπορούμε να δούμε τον κώδικα που περιέχει η συγκεκριμένη βιβλιοθήκη.

```
1  /*In order to enable uip statistics, I created the project-conf.h file*/
2  #define PROJECT_CONF_H_
3
4  /*if UIP_CONF_STATISTICS defined to 1 we enable the uip statistics tool*/
5  #define UIP_CONF_STATISTICS 1
6
```

Σχήμα 5.2: Βιβλιοθήκη ενεργοποίησης του uip statistics

Αφού δημιουργήθηκε αυτή η βιβλιοθήκη, στη συνέχεια έγινε η αναφορά επιπλέον βιβλιοθηκών στον κώδικα του client, έτσι ώστε να μπορέσει να λειτουργήσει το εργαλείο στατιστικής καταμέτρησης. Οι βιβλιοθήκες βρίσκονται στον προορισμό /net/ipv6 και είναι οι εξής: uip.h, uip-ds6.h και uip-udp-packet.h. Έπειτα, προστέθηκαν οι μεταβλητές των παραπάνω βιβλιοθηκών που είναι υπεύθυνες για την καταμέτρηση των πακέτων και μηνυμάτων ελέγχου. Ο client διαμορφώθηκε έτσι ώστε να στείλει μήνυμα προς τον root και αφού λάβει απάντηση, να τυπώσει πόσα πακέτα και μηνύματα ελέγχου έστειλε και έλαβε. Τέλος, όσον αφορά το κομμάτι του κώδικα στην εικόνα 5.3 φαίνεται η τροποποίηση του “Makefile” για να μπορέσει να τρέξει η κλασσική έκδοση RPL, καθώς εξ-ορισμού τρέχει η ελαφριά έκδοση χωρίς αποθήκευση στους κόμβους, ενώ η λειτουργία της αποθήκευσης δεν υποστηρίζεται με τους υπάρχοντες μηχανισμούς.

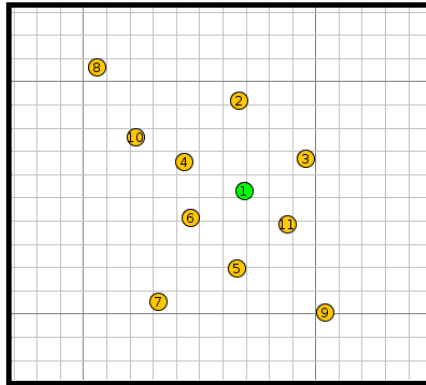
```
1  CONTIKI_PROJECT = udp-client udp-server
2  all: $(CONTIKI_PROJECT)
3
4  CONTIKI=../..
5
6  CFLAGS += -DPROJECT_CONF_H=\"project-conf.h\"
7  MAKE_ROUTING = MAKE_ROUTING_RPL_CLASSIC
8
9  include $(CONTIKI)/Makefile.include
10
```

Σχήμα 5.3: Τροποποίηση του αρχείου Makefile

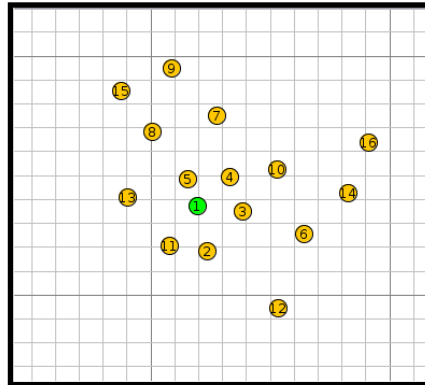
Από τη στιγμή που τελείωσαν οι τροποποιήσεις του κώδικα έπρεπε να επιλεγεί το είδος του υλικού στο οποίο θα προσομοιωθεί ο κώδικας. Τα δύο είδη υλικών που μπορούσαμε να εκτελέσουμε τον κώδικα ήταν ο z1 mote και ο sky mote. Για να επιτευχτεί αυτό, όμως, έπρεπε πρώτα να γίνει μια τροποποίηση της μεταβλητής `NETSTACK_MAX_ROUTE_ENTRIES`, στην βιβλιοθήκη “`contiki-default-conf.h`” στον φάκελο `contiki-ng/os`, από 16 σε 50, έτσι ώστε να επιτρέπονται μέχρι και 50 downward messages από τον root, καθώς εξ-ορισμού υποστηρίζονται μόνο 16. Όταν έγινε λοιπόν αυτή η αλλαγή, ξεκίνησε και το compilation του κώδικα με σκοπό την προσομοίωση στο υλικό του z1 mote.

Στην προκειμένη περίπτωση, εντοπίστηκε το πρώτο πρόβλημα, το οποίο ήταν ότι ο root έστειλε μόνο σε δέκα κόμβους downward messages, ενώ λάμβανε από όλους τους κόμβους upwards messages, και αυτό οφείλεται σε στη μικρού μεγέθους μνήμη που έχει ο z1 mote. Από την στιγμή που βρέθηκε αυτό το πρόβλημα, το compilation του κώδικα δοκιμάστηκε σε τύπο sky mote, καθώς έχει περισσότερη μνήμη, έτσι ώστε να δούμε εάν μπορεί να γίνει σωστά η εκτέλεση του πειράματος. Δυστυχώς, και πάλι η μνήμη δεν ήταν αρκετή έτσι ώστε ο root να χειριστεί downward messages για πενήντα κόμβους, και για αυτό τα downward messages δεν ξεπέρασαν τα 20. Συνεπώς, και λόγω περιορισμού του χρόνου, άλλαξε ο αριθμός των κόμβων στα παραδείγματα που εξετάστηκαν.

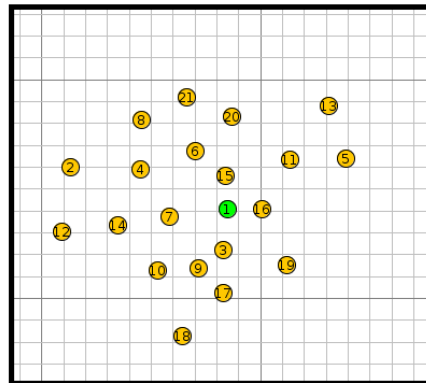
Από την στιγμή που τα downward μηνύματα δεν ξεπέρασαν τα είκοσι, αποφασίσαμε να εκτελέσουμε τα σενάρια μειώνοντας το πλήθος των κόμβων. Τα τρία παραδείγματα των δέκα κόμβων έμειναν ως είχαν, ενώ στο παράδειγμα των εικοσιπέντε κόμβων, μειώσαμε τους κόμβους κατά δέκα, καταλήγοντας σε τρία παραδείγματα των δεκαπέντε κόμβων. Ακόμη μεγαλύτερη μείωση εφαρμόστηκε στο παράδειγμα των πενήντα κόμβων, το οποίο μειώθηκε κατά τριάντα κόμβους, έχοντας στο τέλος τρία παραδείγματα των είκοσι κόμβων (βλ. σχήμα 5.4), δηλαδή τον ανώτατο αριθμό downward μηνυμάτων που μπορέσαμε να φτάσουμε. Για να εξαλειφθούν όσο το δυνατόν περισσότεροι παράγοντες τυχαιότητας και στις τρεις τριάδες παραδειγμάτων, οι κόμβοι είχαν ακριβώς την ίδια θέση στους άξονες των x, y, και z. Η εκτέλεση διήρκησε για το κάθε παράδειγμα 120 λεπτά, έχοντας τον ίδιο αριθμό TX range και INT range στην ρύθμιση transmission ranges του cooja simulator.



10 nodes



15 nodes



20 nodes

Σχήμα 5.4: 10, 15 και 20 κόμβοι στους άξονες x και y

Για την εξαγωγή αποτελεσμάτων, χρησιμοποιήθηκαν οι μετρικές Packet Delivery Ratio και Control Overhead έτσι ώστε να διαπιστώσουμε, με την βοήθεια της εκτέλεσης των σεναρίων, ποια έκδοση του πρωτοκόλλου μπορεί να χειριστεί καλύτερα τον έλεγχο και την διαχείριση των πακέτων, έτσι ώστε να μην υπάρχουν απολεσθέντα πακέτα, δηλαδή πακέτα που χάθηκαν στη διαδρομή. Ξεκινώντας με την πρώτη, το PDR - Packet Delivery Ratio, είναι ο λόγος του αριθμού των UDP πακέτων που έχει λάβει ο κόμβος ($rUDP$) προς τον αριθμό των UDP πακέτων που έχει στείλει ($sUDP$), $PDR = rUDP/sUDP$. Συνεχίζοντας με την δεύτερη μετρική που θα χρησιμοποιηθεί, το CO - Control Overhead, είναι ο λόγος του αθροίσματος των μηνυμάτων ελέγχου (CM) προς το άθροισμα των μηνυμάτων ελέγχου (*Control messages*) μαζί με τον αριθμό των UDP πακέτων που έχει στείλει ($sUDP$) ο κόμβος, $CO = CM/(CM + sUDP)$ [12, 13].

Μετά από την εκτέλεση του κάθε πειραματικού σεναρίου έγινε αποθήκευση των log files, και στη συνέχεια εισάχθηκαν στο εργαλείο Microsoft Excel. Από εκεί υπολογίσαμε τις παραπάνω μετρικές για κάθε κόμβο και μετά τον μέσο όρο των μετρικών για το κάθε σενάριο. Τέλος, εισάγαμε τους μέσους όρους σε διαγράμματα των δύο αξόνων, έτσι ώστε να πραγματοποιηθεί η γραφική παρουσίαση τους για το επόμενο κεφάλαιο.

Κεφάλαιο 6

Πειραματική ανάλυση

Στα προηγούμενα κεφάλαια παρουσιάσαμε τη μεθοδολογία που χρησιμοποιήσαμε για τη διεξαγωγή της διατριβής καθώς και διατριβές συναδέλφων σε συνδυασμό με την θεωρία, η οποία είναι απαραίτητη για την κατανόηση της έρευνας. Στο κεφάλαιο που θα ακολουθήσει, θα αναλύσουμε τα δεδομένα που εξάγαμε κατά την διάρκεια της εκτέλεσης των πειραμάτων και στην συνέχεια τα επεξεργαστήκαμε. Όπως προαναφέραμε, δημιουργήσαμε πέντε σενάρια, με το κάθε σενάριο να αποτελείται από εννιά παραδείγματα. Τα αποτελέσματα των σεναρίων ακολουθούν.

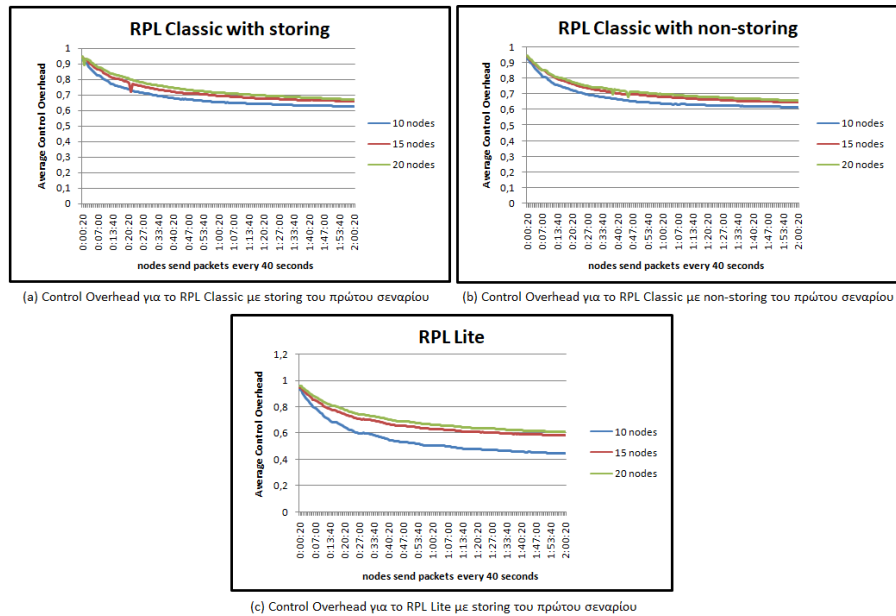
6.1 Πρώτο σενάριο: Συχνή επικοινωνία για διαφορετικές τοπολογίες

Ξεκινώντας με το πρώτο σενάριο το οποίο προσομοιώσαμε, οι κόμβοι προγραμματίστηκαν έτσι ώστε να στέλνουν upward πακέτα κάθε σαράντα δευτερόλεπτο προς τον root αναμένοντας από αυτόν κάποια απάντηση με ένα downward μήνυμα. Για την απλοποίηση της έρευνας, δεν κάνουμε πλήρη έρευνα πόσα πακέτα χάθηκαν κατά την αποστολή τους από τους κόμβους προς τον root και αντίστροφα, αλλά πόσα πακέτα έστειλαν οι κόμβοι και δεν πήραν ποτέ απάντηση. Ο όγκος των δεδομένων που υπέστησαν επεξεργασία σε συνδυασμό με τον χρόνο που καταναλώσαμε στην λύση των προβλημάτων τα οποία αναφέρθηκαν στο κεφάλαιο πέντε δεν μας άφησαν, δυστυχώς, αυτό το περιθώριο.

Αρχικά, βλέποντας το σχήμα 6.1, διακρίνουμε τα διαγράμματα του

Average Control Overhead όσον αφορά το πρώτο σενάριο. Όπως φαίνεται, η έκδοση του RPL Classic με storing (διάγραμμα 6.1.a) και με non-storing λειτουργία (διάγραμμα 6.1.b), έχουν πανομοιότυπα αποτελέσματα με τους μέσους όρους τους να ξεκινούν από το 95% και στο τέλος να κυμαίνονται ανάμεσα σε 60% με 70%. Επιπλέον παρατηρούμε ότι τα παραδείγματα που τρέχουν με non-storing λειτουργία έχουν κατά μέσο όρο 2% μικρότερο Control Overhead από τα παραδείγματα με storing λειτουργία, αντίστοιχα.

Στην έκδοση του RPL Lite (διάγραμμα 6.1.c) από την άλλη πλευρά και όπως ήταν αναμενόμενο, καθώς έχουμε αναφέρει στο τρίτο κεφάλαιο ότι αποστέλει λιγότερα μηνύματα ελέγχου από τη βασική έκδοση, παρατηρούμε μικρότερο Control Overhead. Συγκεκριμένα, σε όλα τα παραδείγματα η μετρική ξεκινά από το 94%, με το παράδειγμα των δέκα κόμβων να καταλήγει στο 44% και τα παραδείγματα των δεκαπέντε και είκοσι κόμβων να καταλήγουν στο 59% και 61% αντίστοιχα.

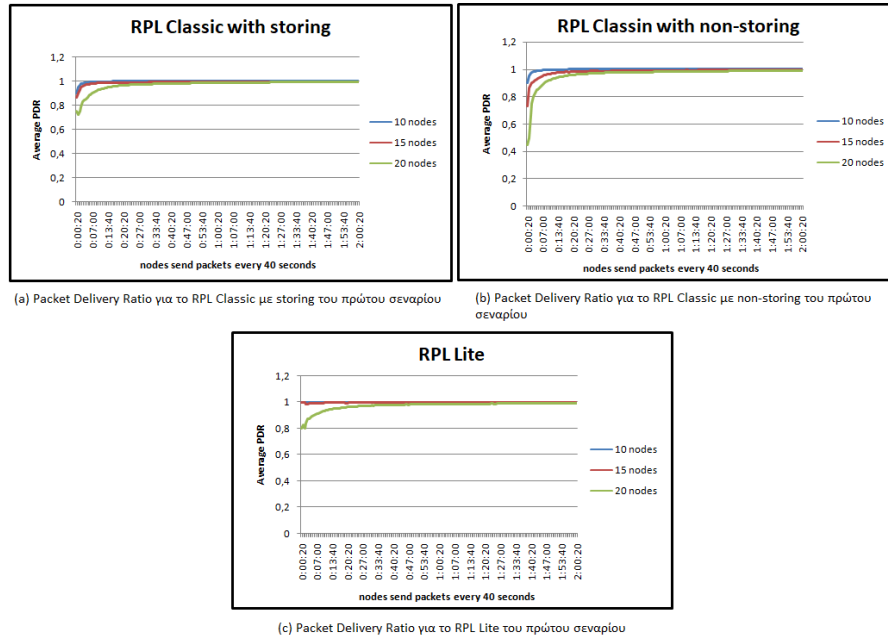


Σχήμα 6.1: Διαγράμματα Μέσου Όρου του Control Overhead πρώτου σεναρίου

Εκτός από την εξέταση των παραδειγμάτων του κάθε σεναρίου με την βοήθεια της μετρικής του Control Overhead, κάνουμε χρήση και της μετρικής του Packet Delivery Ratio. Στο σχήμα 6.2 βλέπουμε τα διαγράμματα των μέσων όρων του Packet Delivery Ratio για τα παραδείγματα του σεναρίου αποστολής πακέτων ανά 40 δευτερόλεπτα. Από αυτά προκύπτουν, σύμφωνα με το διάγραμμα 6.2.a, για το RPL Classic με storing λειτουργία, στα παραδείγματα των δέκα και δεκαπέντε κόμβων, ένα 88-90% έχει κάνει την αμφίδρομη διαδρομή επιτυχώς κατά την εκτέλεση τους, ενώ όσο εκτελούνται βλέπουμε όλο και λιγότερα πακέτα να χάνονται, έως και μηδαμινά από το πέμπτο λεπτό της εκτέλεσης των δύο συγκεκριμένων παραδειγμάτων και μετά. Στην περίπτωση του παραδείγματος των είκοσι κόμβων έχουμε μικρότερο ποσοστό επιτυχίας 75% κατά την έναρξη του, καθώς και απώλειες πακέτων μέχρι το εικοστό λεπτό περίπου της εκτέλεσης των πειραμάτων, με την υπόλοιπη μια ώρα και σαράντα λεπτά να έχουν χαθεί ελάχιστα πακέτα.

Συνεχίζοντας στο διάγραμμα 6.2.b, το οποίο αφορά το RPL Classic με non-storing λειτουργία, για το παράδειγμα των δέκα κόμβων παρατηρούμε και πάλι 90% επιτυχία στην έναρξη του και από τα 5 λεπτά εκτέλεσης και μετά μηδαμινές απώλειες. Στο παράδειγμα των δεκαπέντε κόμβων, η επιτυχία της αμφίδρομης επικοινωνίας ανέρχεται στο 73% με μηδενικές απώλειες από το δέκατο λεπτό και μετά, με το παράδειγμα των είκοσι κόμβων να έχει κατά την έναρξη μόλις 45% επιτυχία και τα πακέτα να συνεχίζουν να χάνονται μέχρι και το πρώτο μισάωρο κατά μέσο όρο.

Φτάνοντας στο διάγραμμα 6.2.c, του RPL Lite, διακρίνουμε ότι για την εκτέλεση του παραδείγματος των δέκα κόμβων δεν έχει χαθεί κανένα πακέτο, 100% επιτυχία καθ' όλη την διάρκεια της εκτέλεσης. Στο παράδειγμα των δεκαπέντε κόμβων, βλέπουμε ότι ξεκινάει με 100% επιτυχία, η οποία πέφτει στο 98% στα πρώτα τρία λεπτά αλλά στη συνέχεια, η εκτέλεση του παραδείγματος συνεχίζεται χωρίς απώλειες, κατά μέσο όρο. Τέλος, για το παράδειγμα των είκοσι κόμβων βλέπουμε ένα ποσοστό επιτυχίας 80% κατά την έναρξη του, ενώ από το λεπτό τριάντα και μετά δεν χάνεται σχεδόν κανένα πακέτο.



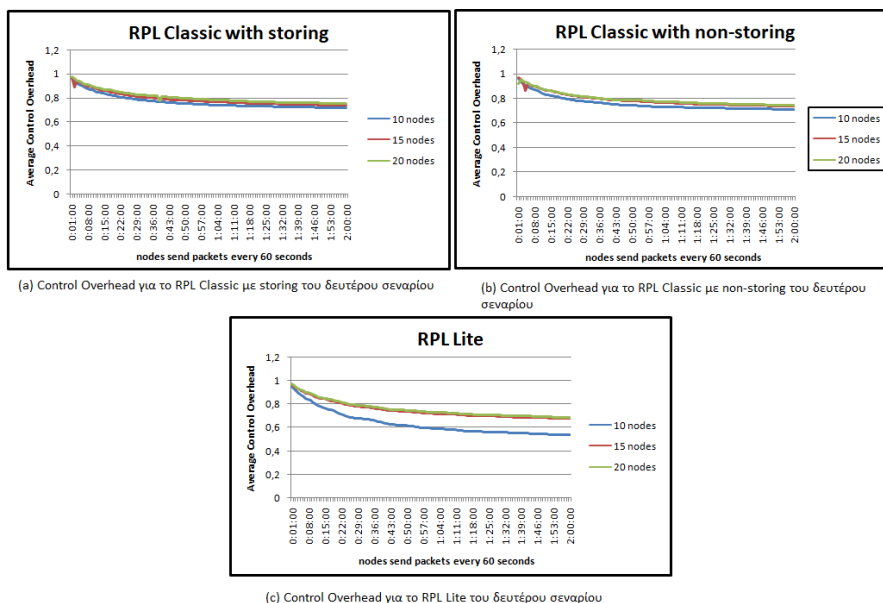
Σχήμα 6.2: Διαγράμματα Μέσου Όρου του Packet Delivery Ratio πρώτου σεναρίου

6.2 Δεύτερο σενάριο: Επικοινωνία ανά λεπτό για διαφορετικές τοπολογίες

Περνάμε στη δεύτερη ενότητα του κεφαλαίου, όπου θα δούμε την συμπεριφορά των πρωτοκόλλων με αποστολή πακέτων κάθε 60 δευτερόλεπτα. Εννοείται ότι από τη στιγμή που μεσολαβεί κάποιο χρονικό διάστημα μεταξύ της αποστολής των πακέτων, υπάρχει ελεγχόμενο φορτίο μέσα στο δίκτυο και φυσικά ο τελικός αριθμός πακέτων που θα έχουν αποσταλεί θα είναι μικρότερος σε σχέση με σενάριο συχνότερης αποστολής πακέτων.

Όπως μπορούμε να δούμε στο σχήμα 6.3, τα διαγράμματα 6.3.a και 6.3.b τα οποία αναφέρονται στο μέσο όρο της μετρικής του Control Overhead των RPL Classic με storing και non-storing λειτουργία αντίστοιχα, στη διάρκεια εκτέλεσης των παραδειγμάτων κινούνται σχεδόν παράλληλα. Και στα δύο διαγράμματα βλέπουμε ο μέσος όρος να ξεκινά από το 97% και να καταλήγει κατά μέσο όρο στο 72%-73%. Στο διάγραμμα 6.3.c για το RPL Lite, όπως και στο πρώτο σενάριο,

έτσι και εδώ όλα τα παραδείγματα ξεκινούν με μέσο όρο 96%, το παράδειγμα των δέκα κόμβων καταλήγει στο 54% με τα παραδείγματα των δεκαπέντε και είκοσι κόμβων να καταλήγουν στο πέρας της εκτέλεσης τους στο 68%.

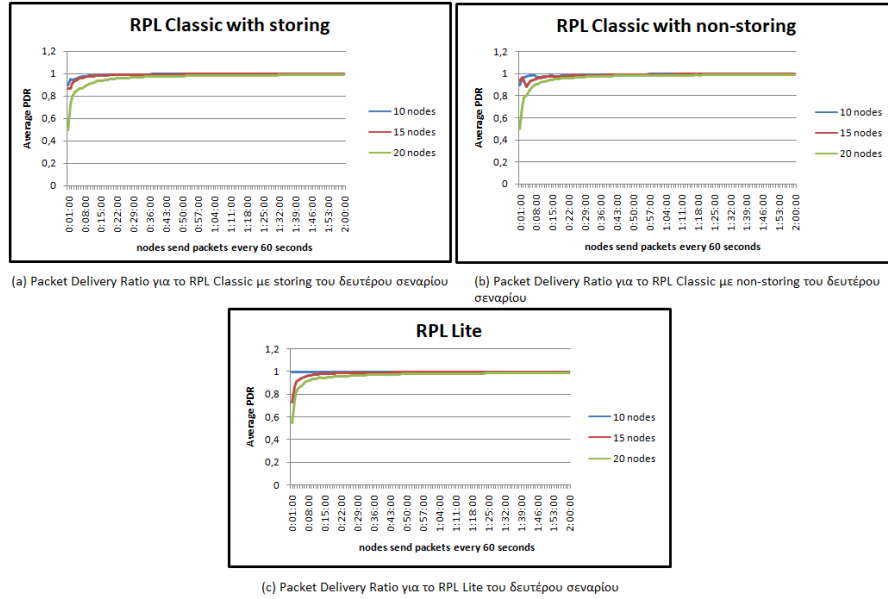


Σχήμα 6.3: Διαγράμματα Μέσου Όρου του Control Overhead δευτέρου σεναρίου

Συνεχίζουμε με την μετρική του Packet Delivery Ratio και τους μέσους όρους όπου απεικονίζονται στο σχήμα 6.4. Από εδώ διακρίνουμε στο διάγραμμα 6.4.a για το PRL Classic με storing λειτουργία ότι, στο παράδειγμα των δέκα κόμβων βλέπουμε μια επανάληψη στον τρόπο συμπεριφοράς με τον μέσο όρο της επιτυχίας να ξεκινάει στο 90% και στο δέκατο λεπτό να έχουν σταματήσει οι απώλειες πακέτων. Το ίδιο σχεδόν βλέπουμε να συμβαίνει και στους 15 κόμβους ενώ στο παράδειγμα των 20 κόμβων μπορούμε να διακρίνουμε ότι το ποσοστό επιτυχίας κατά την έναρξη ανέρχεται στο 50% και ανέρχεται μέχρι τα πρώτα τριάντα λεπτά όπου και οι απώλειες πακέτων έχουν σταματήσει κατά μέσο όρο.

Στο διάγραμμα 6.4.b για το RPL Classic με non-storing λειτουργία, για τους δέκα κόμβους, ενώ φαίνεται ότι θα ακολουθήσει την ίδια συμπεριφορά κατά την έναρξη, στο δέκατο λεπτό παρατηρούμε μια μικρή απώλεια ενώ μετά από ένα δεκάλεπτο βλέπουμε να σταθεροποιείται και πάλι. Στο παράδειγμα των δεκαπέντε κόμβων παρόλο που ξεκινάει με μεγαλύτερο ποσοστό επιτυχίας και από αυτό των δέκα κόμβων στο δεύτερο λεπτό χάνει μεγάλο ποσοστό πακέτων καθώς μειώνεται το ποσοστό επιτυχίας κατά 6% και στο εικοστό λεπτό βλέπουμε να έχει σταθεροποιηθεί. Τέλος, στο παράδειγμα των είκοσι κόμβων, όπως και στο διάγραμμα 6.4.a, έτσι και εδώ η επιτυχία κατά την έναρξη ανέρχεται στο 50% και στα πρώτα εικοσιπέντε με τριάντα λεπτά ομαλοποιείται η κατάσταση.

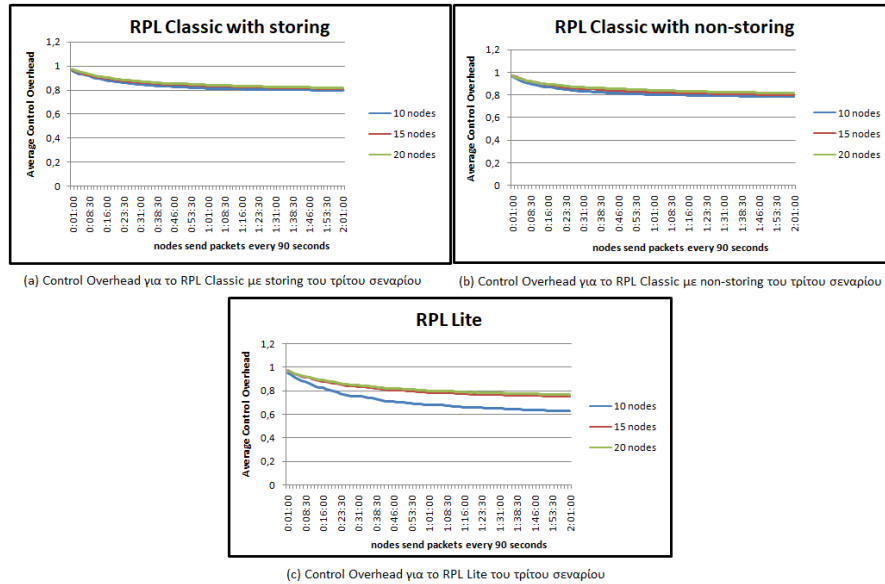
Κλείνουμε την ενότητα 6.2 με το διάγραμμα 6.4.c που αφορά το RPL Lite. Εδώ θα παρατηρήσουμε στο παράδειγμα των δέκα κόμβων να υπάρχει 100% καθ' όλη την διάρκεια της εκτέλεσης. Αντίθετα με τους δέκα κόμβους, στο παράδειγμα των δεκαπέντε κόμβων βλέπουμε το ποσοστό επιτυχίας κατά την εκκίνηση να βρίσκεται στο 73% και μέχρι το δεκατοπέμπτο λεπτό να έχουν σταματήσει η απώλειες. Κλείνοντας με το παράδειγμα των είκοσι κόμβων, κατά την έναρξη έχει ποσοστό επιτυχίας 55%, μεγαλύτερο σε σχέση με την κλασική έκδοση του πρωτοκόλλου που είδαμε στα διαγράμματα 6.4.a και 6.4.b, ενώ μέχρι λεπτό τριάντα φαίνεται να έχουν σταματήσει οι απώλειες πακέτων.



Σχήμα 6.4: Διαγράμματα Μέσου Όρου του Packet Delivery Ratio δευτέρου σεναρίου

6.3 Τρίτο σενάριο: Μέτρια επικοινωνία για διαφορετικές τοπολογίες

Σε αυτή την ενότητα θα περάσουμε στο τρίτο σενάριο με τους κόμβους να στέλνουν δεδομένα κάθε 90 δευτερόλεπτα. Από το σχήμα 6.5 διακρίνουμε ότι στα διαγράμματα 6.5.a και 6.5.b των RPL Classic με storing και non-storing λειτουργία αντίστοιχα, όλα τα παραδείγματα ξεκινούν έχοντας μέσο όρο του Control Overhead το 96% και καταλήγουν στο 80%. Στο διάγραμμα 6.5.c του RPL Lite, όπως και στα διαγράμματα 6.13 και 6.14, όλοι οι κόμβοι έχουν αφετηρία ως μέσο όρο το 96%, με το παράδειγμα των δέκα κόμβων να καταλήγει στο 63% και τα παραδείγματα των δεκαπέντε και είκοσι κόμβων στο 75% και 76% αντίστοιχα.

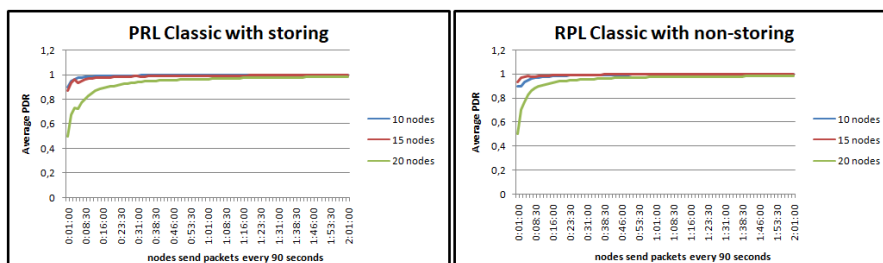


Σχήμα 6.5: Διαγράμματα Μέσου Όρου του Control Overhead τρίτου σεναρίου

Αλλάζουμε τώρα στο σχήμα 6.6 όπου και θα δούμε το μέσο όρο του Packet Delivery Ratio. Βλέπουμε λοιπόν στο διάγραμμα 6.6.a ότι στα παραδείγματα των δέκα και δεκαπέντε κόμβων έχουμε εκκίνηση της εκτέλεσης τους 90%, με το παράδειγμα των 15 κόμβων ενώ ανεβαίνει ποσοστιαία, στο έβδομο λεπτό να υπάρχει μια πτώση της απόδοσης. Στο παράδειγμα των δέκα κόμβων βλέπουμε να έχει σταματήσει εντελώς η απώλεια πακέτων στο έκτο λεπτό με το παράδειγμα των δεκαπέντε κόμβων να έχει ανακτηθεί στο όγδοο λεπτό και τριακοστό δευτερόλεπτο. Τέλος, στους είκοσι κόμβους παρατηρούμε ότι το PDR ξεκινάει από το 50%, με μια μικρή πτώση στο έβδομο λεπτό ενώ οι απώλειες σταματούν να υφίστανται στην πρώτη ώρα της εκτέλεσης του.

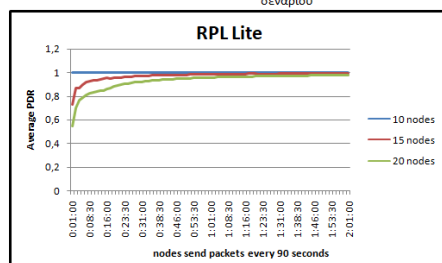
Στο διάγραμμα 6.6.b, βλέπουμε για πρώτη φορά το παράδειγμα των 15 κόμβων να έχει μεγαλύτερο ποσοστό επιτυχίας κατά την έναρξη σε σχέση με αυτό των δέκα κόμβων. Στους είκοσι κόμβους βλέπουμε και πάλι μια εκκίνηση με 50% επιτυχία παύση απώλειας πακέτων στα 55 λεπτά. Τέλος, στο διάγραμμα 6.6.c, οι δέκα κόμβοι έχουν χάσει μηδέν πακέτα, οι δεκαπέντε κόμβοι εκκινούν από το 73% και ο μέσος

όρος των κόμβων από το 55%, με λίγες ενδιάμεσες απώλειες και στα δύο.



(a) Packet Delivery Ratio για το RPL Classic με storing του τρίτου σεναρίου

(b) Packet Delivery Ratio για το RPL Classic με non-storing του τρίτου σεναρίου



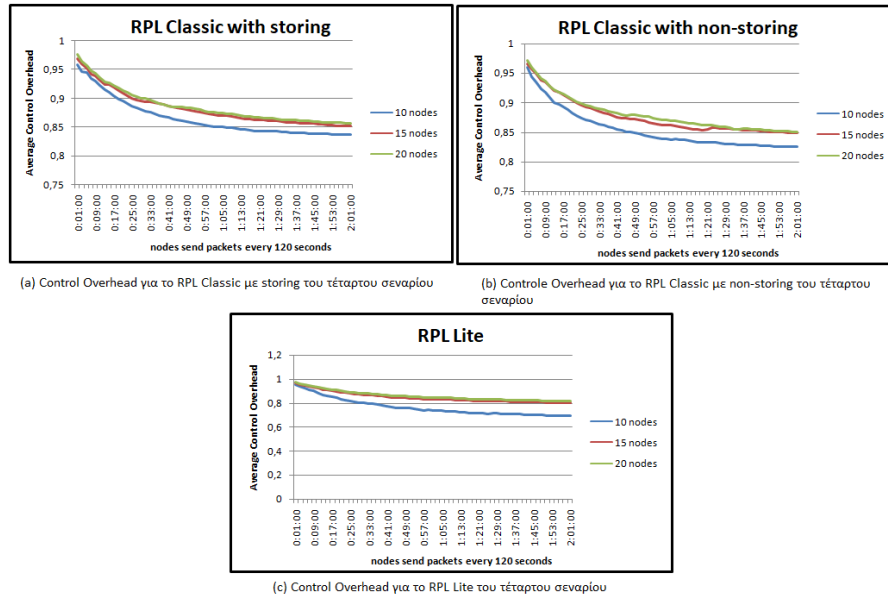
(c) Packet Delivery Ratio για το RPL Lite του τρίτου σεναρίου

Σχήμα 6.6: Διαγράμματα Μέσου Όρου του Packet Delivery Ratio τρίτου σεναρίου

6.4 Τέταρτο σενάριο: Αραιή επικοινωνία για διαφορετικές τοπολογίες

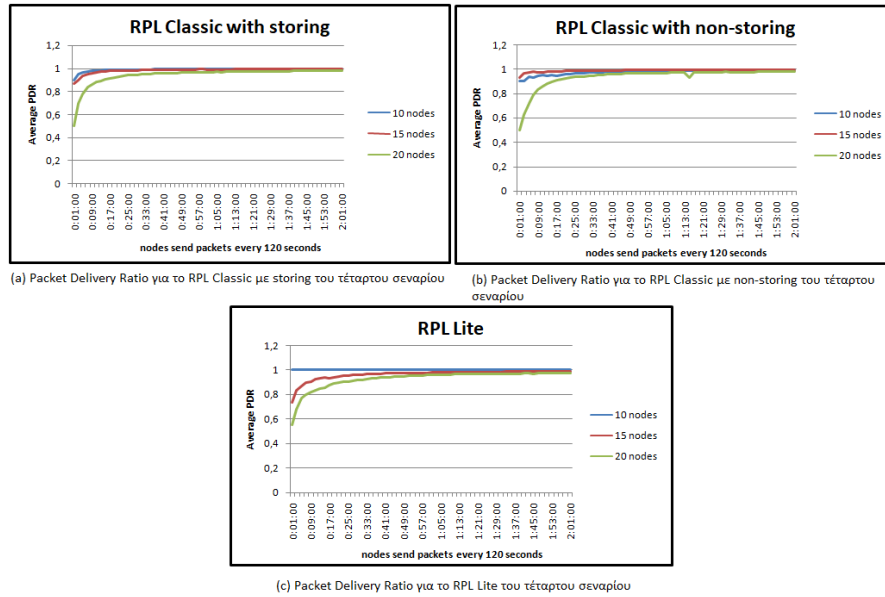
Στην ενότητα που ακολουθεί, περνάμε στο τέταρτο σενάριο μας, την αποστολή πακέτων κάθε 120 δευτερόλεπτα. Όπως προείπαμε, όσο αυξάνουμε τα λεπτά που μεσολαβούν από πακέτο σε πακέτο, έχουμε και μείωση των συνολικών πακέτων που ανταλλάσσονται μεταξύ των κόμβων. Παρατηρούμε ότι όσο πιο πολύ αυξάνεται, τόσο πιο πολύ αυξάνεται και το Control Overhead, κάτι που δεν θέλουμε να συμβαίνει. Σε αυτό το σημείο περνάμε στο σχήμα 6.7, στο διάγραμμα 6.7.a για την κλασική έκδοση του πρωτοκόλλου με λειτουργία αποθήκευσης. Όπως μπορούμε να δούμε, κατά την έναρξη, οι δέκα κόμβοι ξεκινούν με Control Overhead στο 96%, οι δεκαπέντε στο 97% και οι είκοσι στο 98% ενώ καταλήγουν στο τέλος στο 83% το παράδειγμα των δέκα και στο 85% το παράδειγμα δεκαπέντε και είκοσι.

Στο διάγραμμα 6.7.b παρατηρούμε για την κλασική έκδοση με την χωρίς αποθήκευση λειτουργία τα ίδια ποσοστά όπως και στο 6.7.a. Στο 6.7.c, για την ελαφριά έκδοση του πρωτοκόλλου βλέπουμε ότι κατά την έναρξη υπάρχει το ίδιο ποσοστό Control Overhead με τα προηγούμενα δύο πειράματα αλλά οι δέκα κόμβοι καταλήγουν στο 70% και τα παραδείγματα των δεκαπέντε και είκοσι κόμβων στο 80%.



Σχήμα 6.7: Διαγράμματα Μέσου Όρου του Control Overhead τετάρτου σεναρίου

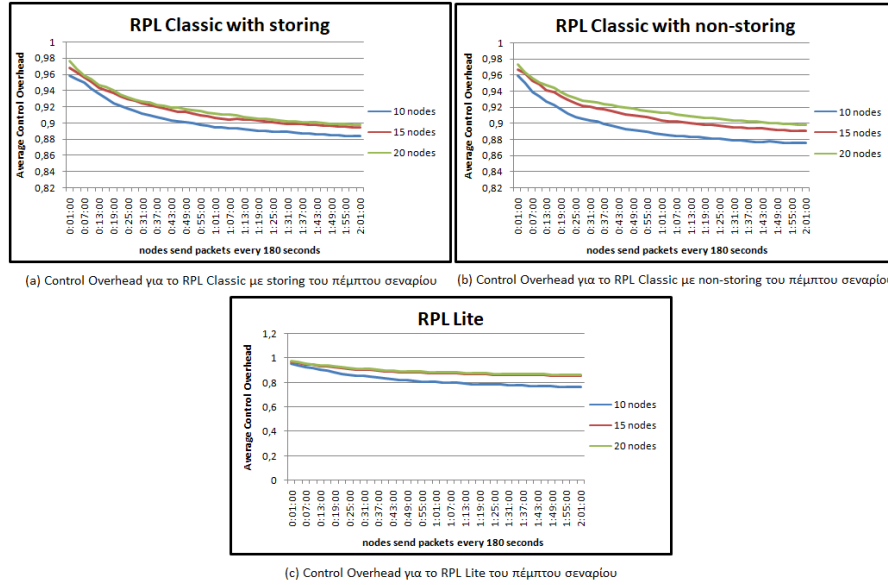
Στην δεύτερη μετρική πάλι, όπου απεικονίζονται και τα αποτελέσματα στην εικόνα 6.8, 6.8.a, 6.8.b και 6.8.c των RPL Classic με λειτουργία αποθήκευσης και χωρίς αποθήκευση καθώς στο RPL Lite, διακρίνουμε παρόμοια αποτελέσματα με το τρίτο σενάριο. Η μόνη διαφορά εδώ είναι ότι μεταξύ των πειραμάτων υπάρχουν λιγότερες ενδιάμεσες απώλειες μέχρι να φτάσουν το σημείο που αγγίζουν το 99% επιτυχίας αποστολής πακέτων. Τέλος, για ακόμη μια φορά στο παράδειγμα των δέκα κόμβων του RPL Lite έχουμε καμία απώλεια πακέτων, ενώ στο παράδειγμα των είκοσι κόμβων το 99% το φτάνει στην μια ώρα όπως και η κλασική έκδοση με λειτουργία αποθήκευσης, κάτι που δεν συμβαίνει στην κλασική έκδοση χωρίς αποθήκευση. Συνεπώς, για τους είκοσι κόμβους στο διάγραμμα 6.8.b έχουμε 5% περισσότερες απώλειες σε σχέση με το 6.8.c αλλά οι απώλειες σταματούν στην μισή ώρα από την έναρξη και όχι στην μια ώρα όπως συμβαίνει στην ελαφριά έκδοση.



Σχήμα 6.8: Διαγράμματα Μέσου Όρου του Packet Delivery Ratio τε-
τάρτου σεναρίου

6.5 Πέμπτο σενάριο: Πολύ αραιή επικοινωνία για διαφορετικές τοπολογίες

Τελευταίο είναι το πέμπτο σενάριο που διεξήγαμε, με τους μέσους όρους του Control Overhead να εμφανίζονται στην εικόνα 6.9. Σε αυτό οι κόμβοι προγραμματίστηκαν να στέλνουν πακέτα κάθε 180 δευτερόλεπτα ή τρία λεπτά. Στα διαγράμματα 6.9.a και 6.9.b του RPL Classic με storing και non-storing λειτουργία διακρίνουμε ότι στα παραδείγματα δέκα, δεκαπέντε και είκοσι κόμβων ο μέσος όρος του Control Overhead ξεκινά από 96%, 97% και 98% αντίστοιχα και καταλήγει στο 88%, 89% και 90%. Στο διάγραμμα 6.9.c του RPL Lite, τα τρία παραδείγματα ξεκινούν από το 96%, 97% και 98% όπως και στα προηγούμενα αλλά καταλήγουν 76% 85% και 86% μέσο όρο.

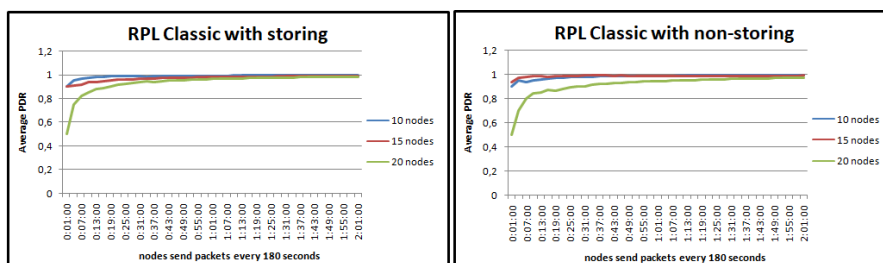


Σχήμα 6.9: Διαγράμματα Μέσου Όρου του Control Overhead πέμπτου σεναρίου

Τελειώνουμε με την εικόνα 6.10 για τους μέσους όρους του Packet Delivery Ratio. Βλέπουμε ότι στην 6.10.a για το RPL Classic με storing λειτουργία η επιτυχία αποστολής πακέτων βρίσκεται στο 90% κατά την έναρξη για τα παραδείγματα δέκα και δεκαπέντε κόμβων και φτάνουν στο 99% επιτυχίας δεκατο τρίτο και τριακοστό λεπτό αντίστοιχα. Το παράδειγμα των 20 κόμβων βλέπουμε κατά την εκκίνηση να υπάρχει κατά μέσο όρο 50% PDR και μέχρι την πρώτη ώρα έχει φτάσει στο 99%.

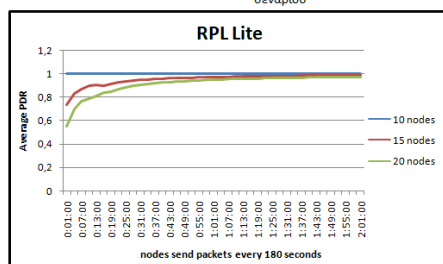
Στο διάγραμμα 6.10.b για ακόμη μια φορά βλέπουμε η απόδοση του παραδείγματος των δεκαπέντε κόμβων να ξεπερνά ακόμη και αυτό των δέκα κόμβων για την κλασική έκδοση χωρίς λειτουργία αποθήκευσης. Στη περίπτωση των είκοσι κόμβων βλέπουμε ότι όχι μόνο ξεκινάει με 50% μέσο όρο αλλά μέχρι και την μια ώρα και τριάντα λεπτά χάνονται πακέτα. Κλείνοντας με το 6.10.c για το RPL Lite, κλασικά στο παράδειγμα των δέκα κόμβων δεν χάσαμε κανένα πακέτο. Οι δεκαπέντε κόμβοι με την ελαφριά έκδοση αποδίδουν και πάλι χειρότερη από την κλασική έκδοση ενώ στους είκοσι κόμβους έχουμε πάλι κατά την έναρξη 55% επιτυχίας αποστολής πακέτων

και στην μια ώρα και δέκα λεπτά βρισκόμαστε στο 99% επιτυχίας αποστολής πακέτων.



(a) Packet Delivery Ratio για το RPL Classic με storing του πέμπτου σεναρίου

(b) Packet Delivery Ratio για το RPL Classic με non-storing του πέμπτου σεναρίου



(c) Packet Delivery Ratio για το RPL Lite του πέμπτου σεναρίου

Σχήμα 6.10: Διαγράμματα Μέσου Όρου του Packet Delivery Ratio πέμπτου σεναρίου

Κεφάλαιο 7

Συμπεράσματα

Στο κεφάλαιο που ακολουθεί, θα εξάγουμε τα βασικά συμπεράσματα σύμφωνα με τα δεδομένα όπου παρουσιάστηκαν στο προηγούμενο κεφάλαιο. Σε γενικές γραμμές επιθυμούμε από ένα δίκτυο να έχει όσο το δυνατό μικρότερο Overhead και ταυτόχρονα όσο το δυνατό μεγαλύτερο Packet Delivery Ratio, έτσι ώστε να μην υπερφορτώνεται το δίκτυο μας με μηνύματα ελέγχου, συγκριτικά με τα πακέτα που έχουμε στείλει και όλα τα πακέτα να φτάνουν στον προορισμό τους. Έτσι μπορούμε να έχουμε στην διάθεση μας ένα δίκτυο αξιόπιστο, το οποίο δεν θα έχει απώλειες, κάτι που είναι πολύ δύσκολο να συμβεί, έως και ακατόρθωτο.

Στη διατριβή μας προκύπτει ότι το RPL Lite και για τα πέντε σενάρια που διεξήγαμε, από την άποψη του Control Overhead έχει την καλύτερη απόδοση, και είναι φυσιολογικό καθώς έχει σχεδιαστεί να λειτουργεί με πολύ λιγότερα μηνύματα ελέγχου σε σχέση με το RPL Classic. Το ίδιο συμπέρασμα βγαίνει και κατά την ανάλυση του Packet Delivery Ratio για το πρώτο σενάριο που εξετάσαμε. Σύμφωνα με τους μέσους όρους των διαγραμμάτων μας, το RPL Lite είχε την μεγαλύτερη αναλογία παράδοσης πακέτων και από το RPL Classic με storing και non-storing λειτουργία.

Στο δεύτερο σενάριο, παρατηρήσαμε κάτι το παράδοξο στην μετρική του Packet Delivery Ratio. Ενώ για τους δέκα και είκοσι κόμβους το RPL Lite είχε την καλύτερη απόδοση, το RPL Classic με non-storing λειτουργία είχε την καλύτερη απόδοση στο παράδειγμα των δεκαπέντε κόμβων. Και ενώ στο δεύτερο σενάριο παρατηρήθηκε για πρώτη φορά, από το τρίτο σενάριο μέχρι και το πέμπτο το παρα-

τηρούμε και μάλιστα με μεγάλη διαφορά. Τέλος, το RPL Classic με storing λειτουργία είχε την χειρότερη απόδοση, καθώς είχε το μεγαλύτερο Control Overhead αλλά και το Packet Delivery Ratio.

Το RPL Classic με non-storing λειτουργία από άποψη Control Overhead είχε παρόμοιες τιμές με την κλασική έκδοση με λειτουργία αποθήκευσης, οπότε και αυτό σύμφωνα με την μετρική αυτή κατατάσσεται να έχει μεγαλύτερο Overhead από την ελαφριά έκδοση. Στο μόνο πράγμα που υπερτερεί στα τέσσερα από τα πέντε σενάρια στα πειράματα που διεξήγαμε είναι η απόδοση του σύμφωνα με Packet Delivery Ratio για το παράδειγμα των δεκαπέντε κόμβων. Σε όλα υπόλοιπα και αυτό είχε από τις χειρότερες αποδόσεις.

Συμπεραίνουμε ότι το RPL Lite είναι καλύτερο σε σχέση απόδοσης από το RPL Classic είτε με, είτε χωρίς λειτουργία αποθήκευσης για το πρώτο σενάριο, το οποίο ήταν και το πιο απαιτητικό καθώς υπήρχε πολύ μεγάλο φορτίο πακέτων μέσα στο δίκτυο από την στιγμή που οι κόμβοι τα έστελναν κάθε 40 δευτερόλεπτα. Και στα υπόλοιπα σενάρια από άποψη του Control Overhead είναι το καλύτερο και εάν εξαιρέσουμε το παράδειγμα των δεκαπέντε κόμβων σε όλα τα υπόλοιπα ήταν αποδοτικότερο σύμφωνα με το Packet Delivery Ratio. Τέλος, το πιο σημαντικό από όλα, το οποίο πρέπει να αναφερθεί είναι ότι το RPL Lite στους δέκα κόμβους για όλα τα σενάρια μας δεν έχασε πακέτα, και όλα παραδόθηκαν επιτυχώς.

Κεφάλαιο 8

Προτάσεις για μελλοντικές μελέτες

Σκοπός της ερευνητικής εργασίας ήταν να συγκρίνουμε διαφορετικές εκδοχές του πρωτοκόλλου RPL, έτσι ώστε να μπορέσουμε να δούμε τι μπορούν να κάνουν με τις εργοστασιακές τους ρυθμίσεις. Στόχος αυτού του τελευταίου κεφαλαίου είναι η πρόταση ιδεών, έτσι ώστε, κάποιος ερευνητής μελλοντικά να συνεχίσει αυτό το έργο με άλλες παρόμοιες έρευνες, με σκοπό να εμπλουτιστούν οι γνώσεις μας πάνω σε αυτό το καινούργιο εργαλείο που ονομάζεται Contiki-NG, καθώς και σε αυτή την νέα πρόταση πρωτοκόλλου, την Lite έκδοση του RPL.

Ένα βασικό στόχο που προσπαθήσαμε να επιδιώξουμε με την παρούσα έρευνα, αλλά δεν μπορέσαμε να τον πετύχουμε λόγω αδυναμίας υλικού, και αναφερόμαστε στην μικρή ποσότητα μνήμης των mote, είναι να εκτελεστούν τα παραπάνω πειράματα με περισσότερους κόμβους. Αυτό θα μπορούσε να επιτευχθεί είτε με την εύρεση συμβατών mote που μπορούν να εξομοιωθούν στον Cooja Simulator του Contiki-NG, είτε με μια ιδέα που πρότεινε ο συνάδελφος κ. Γιώργος Βιολέτας, προσαρμόζοντας έτσι τον κώδικα με σκοπό κάθε mote εκτός από τον client, να “τρέχει” στο υπόβαθρο και ένας root, ενώ στον root να “τρέχει” στο υπόβαθρο και ένας client. Με αυτό τον τρόπο κάνουμε τους πελάτες να στέλνουν στην ρίζα upward μηνύματα, αλλά αντί ο root να απαντάει με downward μηνύματα πίσω στους πελάτες, θα απαντάει ως client στον κάθε root που τρέχει σε κάθε πελάτη, και με αυτό τον τρόπο θα υπάρχουν μόνο upward μηνύματα μέσα στο DODAG. Αυτή η ιδέα προέκυψε από το γεγονός ότι, ενώ υπήρχε θέμα

μνήμης στα downward, δεν υπήρχε κανένα θέμα στα upward, καθώς ο root έλαβε μέχρι και 50 μηνύματα από τους πελάτες αλλά επέλεξε να απαντήσει μόνο σε 20 λόγο περιορισμού της μνήμης.

Εκτός από το παραπάνω, μπορεί κάποιος είτε εάν εφαρμοστεί η παραπάνω ιδέα, είτε σε συνέχεια της παρούσας έρευνας, είτε σε συνδυασμό της ιδέας που θα προταθεί τώρα με την παραπάνω, να προσπαθήσουμε να τροποποιήσουμε τις διάφορες παραμέτρους στις διάφορες βιβλιοθήκες του Cooja. Συγκεκριμένα, μπορούν να αλλάξουν παράμετροι που έχουν να κάνουν με το Trickle Timer ή ακόμη και να δοκιμαστούν και καινούργιοι αλγόριθμοι σε συνδυασμό με τις δύο εκδόσεις πρωτοκόλλων, όπως για παράδειγμα του Dynamic Double Trickle Timer Algorithm που είδαμε στο [1]. Μια άλλη ιδέα είναι η ενσωμάτωση εκτός από σταθερούς και κινητούς κόμβους ή μόνο κινητούς κόμβους [4, 6] έτσι ώστε να μπορέσουμε να δούμε κατά πόσο αποδοτικές είναι οι δύο εκδόσεις του RPL. Σαφώς, ο κάθε συνδυασμός των παραπάνω είναι δυνατός. Τέλος, ένα ωραίο πείραμα θα ήταν να δούμε την αντίδραση του RPL Lite κάτω από DODAG attack, μια ιδέα που εμπνεύστηκε από την ανάγνωση των ερευνών [7, 8].

Εν κατακλείδι, η διεξαγωγή της παραπάνω διατριβής ήταν μια διαδικασία η οποία άξιζε και τον χρόνο και τον κόπο, παρόλα τα προβλήματα, τα οποία είτε μπόρεσαν να λυθούν, είτε δεν λύθηκαν και αναγράφονται σε αυτό το κεφάλαιο ως μελλοντικές μελέτες και τροφή για σκέψη. Οι γνώσεις που βγήκαν από την έρευνα αυτή συνεισφέρουν στην ακαδημαϊκή και ερευνητική κοινότητα, καθώς μελλοντικά μπορεί να βοηθήσουν στην δημιουργία ασφαλέστερων και αξιόπιστων εφαρμογών πάνω στην τεχνολογία του Internet of Things.

Βιβλιογραφία

- [1] Akyildiz, I. F., Su, W., Sankarasubramaniam, Y., Cayirci, E. (2002). *Wireless sensor networks: a survey*. Computer networks, 38(4), 393-422.
- [2] Mainwaring, A., Culler, D., Polastre, J., Szewczyk, R., Anderson, J. (2002, September). *Wireless sensor networks for habitat monitoring*. In Proceedings of the 1st ACM international workshop on Wireless sensor networks and applications (pp. 88-97).
- [3] Tang, W., Ma, X., Huang, J., Wei, J. (2016). *Toward improved RPL: A congestion avoidance multipath routing protocol with time factor for wireless sensor networks*. Journal of Sensors, 2016.
- [4] Hwang, R. H., Peng, M. C., Wu, C. Y., Abimannan, S. (2020). *A novel RPL-based multicast routing mechanism for wireless sensor networks*. International Journal of Ad Hoc and Ubiquitous Computing, 33(2), 122-131.
- [5] Thubert, P. (2012). *Objective function zero for the routing protocol for low-power and lossy networks (RPL)*.
- [6] Kim, H. S., Ko, J., Culler, D. E., Paek, J. (2017). *Challenging the IPv6 routing protocol for low-power and lossy networks (RPL): A survey*. IEEE Communications Surveys and Tutorials, 19(4), 2502-2525.
- [7] Kim, H. S., Kim, H., Paek, J., Bahk, S. (2016). *Load balancing under heavy traffic in RPL routing protocol for low power and lossy networks*. IEEE Transactions on Mobile Computing, 16(4), 964-979.
- [8] Ghaleb, B., Al-Dubai, A. Y., Ekonomou, E., Alsarhan, A., Nasser, Y., Mackenzie, L. M., Boukerche, A. (2018). *A survey of limitations and enhancements of the ipv6 routing protocol for low-power and*

- lossy networks: A focus on core operations*. IEEE Communications Surveys and Tutorials, 21(2), 1607-1635.
- [9] Oliveira, A., Vazão, T. (2016). *Low-power and lossy networks under mobility: A survey*. Computer networks, 107, 339-352.
 - [10] Zhao, M., Ho, I. W. H., Chong, P. H. J. (2016). *An energy-efficient region-based RPL routing protocol for low-power and lossy networks*. IEEE Internet of Things Journal, 3(6), 1319-1333.
 - [11] Sanshi, S., Jaidhar, C. D. (2019). *Enhanced mobility aware routing protocol for low power and lossy networks*. Wireless Networks, 25(4), 1641-1655.
 - [12] Violettas, G., Petridou, S., Mamatas, L. (2018, December). *Routing under heterogeneity and mobility for the Internet of Things: a centralized control approach*. In 2018 IEEE Global Communications Conference (GLOBECOM) (pp. 1-7). IEEE.
 - [13] Violettas, G., Petridou, S., Mamatas, L. (2019). *Evolutionary Software Defined Networking-Inspired Routing Control Strategies for the Internet of Things*. IEEE Access, 7, 132173-132192.
 - [14] Ko, J., Jeong, J., Park, J., Jun, J. A., Gnawali, O., Paek, J. (2015). *DualMOP-RPL: Supporting multiple modes of downward routing in a single RPL network*. ACM Transactions on Sensor Networks (TOSN), 11(2), 1-20.
 - [15] Gaddour, O., Koubâa, A. (2012). *RPL in a nutshell: A survey*. Computer Networks, 56(14), 3163-3178.
 - [16] Clausen, T., Herberg, U., Philipp, M. (2011, October). *A critical evaluation of the IPv6 routing protocol for low power and lossy networks (RPL)*. In 2011 IEEE 7th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob) (pp. 365-372). IEEE.
 - [17] Vasseur, J., Agarwal, N., Hui, J., Shelby, Z., Bertrand, P., Chauvenet, C. (2011). *RPL: The IP routing protocol designed for low power and lossy networks*. Internet Protocol for Smart Objects (IPSO) Alliance, 36.

- [18] Iova, O., Picco, P., Istomin, T., Kiraly, C. (2016). *Rpl: The routing standard for the internet of things... or is it?*. IEEE Communications Magazine, 54(12), 16-22.
- [19] Kim, H. S., Cho, H., Kim, H., Bahk, S. (2017). *DT-RPL: Diverse bidirectional traffic delivery through RPL routing protocol in low power and lossy networks*. Computer Networks, 126, 150-161.
- [20] Parasuram, A., Culler, D., Katz, R. (2016). *An analysis of the RPL routing standard for low power and lossy networks*. Electrical Engineering and Computer Sciences University of California at Berkeley.
- [21] Mulligan, G. (2007, June). *The 6LoWPAN architecture*. In Proceedings of the 4th workshop on Embedded networked sensors (pp. 78-82).
- [22] Dujovne, D., Watteyne, T., Vilajosana, X., Thubert, P. (2014). *6TiSCH: deterministic IP-enabled industrial internet (of things)*. IEEE Communications Magazine, 52(12), 36-41.
- [23] Shelby, Z., Hartke, K., Bormann, C. (2014). *The constrained application protocol (CoAP)*.
- [24] Thangavel, D., Ma, X., Valera, A., Tan, H. X., Tan, C. K. Y. (2014, April). *Performance evaluation of MQTT and CoAP via a common middleware*. In 2014 IEEE ninth international conference on intelligent sensors, sensor networks and information processing (ISSNIP) (pp. 1-6). IEEE.
- [25] Dunkels, A., Gronvall, B., Voigt, T. (2004, November). *Contiki-a lightweight and flexible operating system for tiny networked sensors*. In 29th annual IEEE international conference on local computer networks (pp. 455-462). IEEE.
- [26] Yassein, M. B., Hmeidi, I., Shehadeh, H., Yaseen, W. B., Masadeh, E. A., Mardini, W., Baker, Q. B. (2019). *Performance Evaluation of "Dynamic Double Trickle Timer Algorithm" in RPL for Internet of Things (IoT)*.
- [27] Qasem, M., Altawssi, H., Yassien, M. B., Al-Dubai, A. (2015, October). *Performance evaluation of RPL objective functions*. In 2015 IEEE International Conference on Computer and Information

Technology; Ubiquitous Computing and Communications; Dependable, Autonomic and Secure Computing; Pervasive Intelligence and Computing (pp. 1606-1613). IEEE.

- [28] Kharrufa, H., Al-Kashoash, H., Al-Nidawi, Y., Mosquera, M. Q., Kemp, A. H. (2017, February). *Dynamic RPL for multi-hop routing in IoT applications*. In 2017 13th Annual Conference on wireless on-demand network systems and services (WONS) (pp. 100-103). IEEE.
- [29] Lee, K. C., Sudhaakar, R., Ning, J., Dai, L., Addepalli, S., Vasseur, J. P., Gerla, M. (2012). *A comprehensive evaluation of rpl under mobility*. International Journal of vehicular technology, 2012.
- [30] Lodhi, M. A., Rehman, A., Khan, M. M., Hussain, F. B. (2015, August). *Multiple path RPL for low power lossy networks*. In 2015 IEEE Asia Pacific Conference on Wireless and Mobile (APWiMob) (pp. 279-284). IEEE.
- [31] Tian, B., Hou, K. M., Shi, H., Liu, X., Diao, X., Li, J., Chanet, J. P. (2013, June). *Application of modified RPL under VANET-WSN communication architecture*. In 2013 international conference on computational and information sciences (pp. 1467-1470). IEEE.
- [32] Antoni, L., C., Alvaro, V., Marco, Z., Antoine, B., Ermanno, P. (2016) *Internet of Things In 5 Days*
- [33] Kavyashree, E. D. (2018). *6LoWPAN network using Contiki operating system*. In 3rd National Conference on Image Processing, Computing, Communication, Networking and Data Analytics (p. 300).
- [34] Velinov, A., Mileva, A. (2016). *Running and testing applications for Contiki OS using Cooja simulator*.
- [35] Kurniawan, A. (2018). *Practical Contiki-NG: Programming for Wireless Sensor Networks*. Apress.
- [36] Ribera, E. G., Alvarez, B. M., Samuel, C., Ioulianou, P. P., Vassilakis, V. G. (2020, July). *Heartbeat-based detection of blackhole and greyhole attacks in RPL networks*. In 12th IEEE/IET International Symposium on Communication Systems, Networks and Digital Signal Processing (CSNDSP).

[37] *<https://github.com/contiki-os/contiki>*

[38] *<https://github.com/contiki-ng/contiki-ng>*